

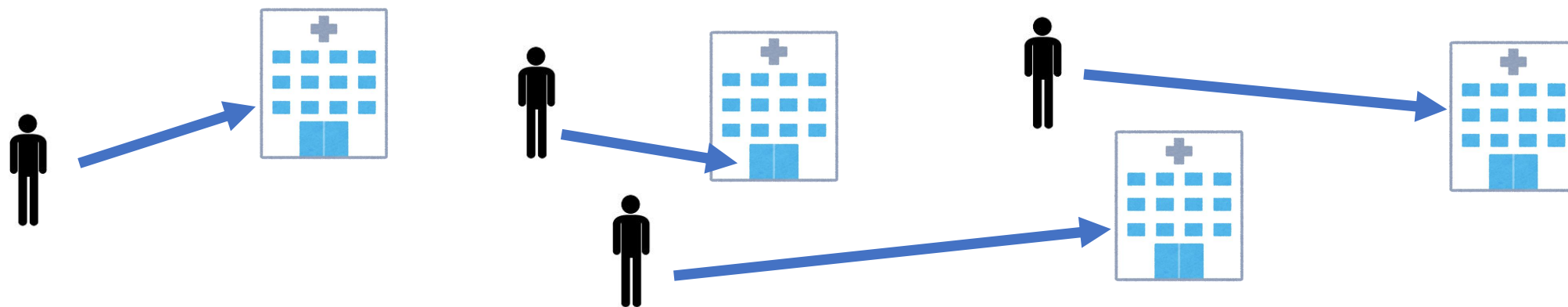
オンラインメトリックマッチング とその拡張

2026/08/17 JCCA-DMIA 2026 @金沢

東京科学大学 澄田研究室 D3

原田 翼

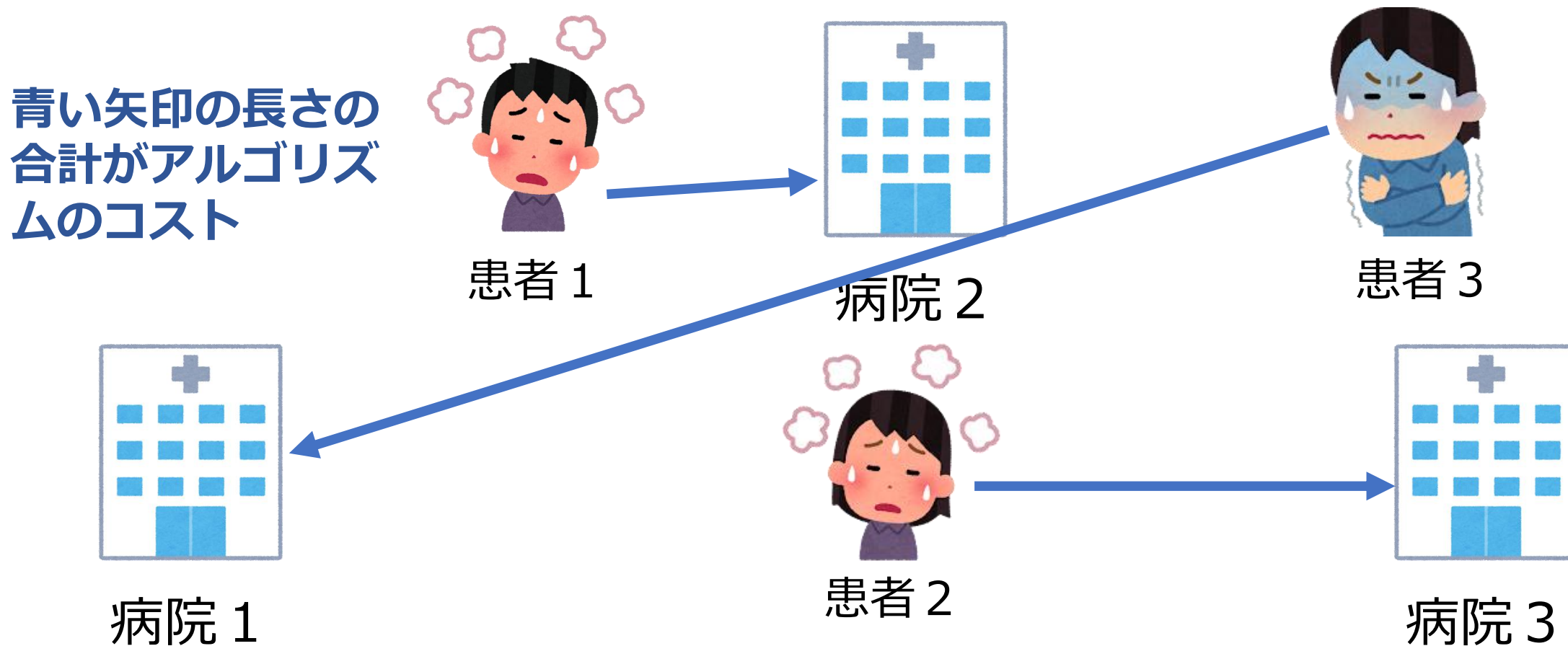
オンラインメトリックマッチング



- n 棟の病院がある
- 急患の患者を直ちに診察中の患者がいない病院に搬送する
- 一度決めた搬送先を後から変更することは出来ない

目的: 患者と割当先の病院間の距離の総和（“→”の長さの合計）を最小化

例 1 : 逐次的な割り当て

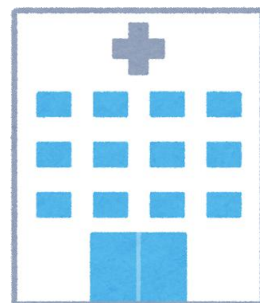


例 2 : 最善の割り当て (結果論)

こう割り当てるのが
最善だった...



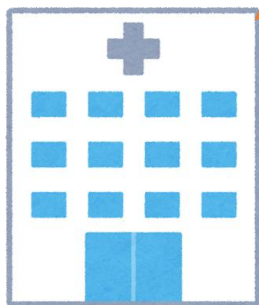
患者 1



病院 2



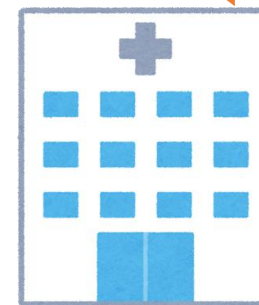
患者 3



病院 1



患者 2



病院 3

問題設定

オンラインメトリックマッチング : $OMM(n)$

Online Metric Matching

入力	距離空間 $\mathcal{M} = (X, d)$ n 個のサーバ $S = \{s_1, s_2, \dots, s_n\} \subseteq X$ リクエスト列 $\sigma = r_1 r_2 \dots r_n \in X^n$ (※リクエストは r_1 から r_n まで逐次的に与えられる)
出力	サーバ S とリクエスト $R = \{r_1, \dots, r_n\}$ の完全マッチング M
制約	異なるリクエストは異なるサーバに割り当てる r_i のマッチ先は S と $r_1, \dots, r_{i-1}$ のみに依存して決定 マッチ先を後から変更しない
最小化	出力 M におけるリクエストとサーバの距離の総和 : $\sum_{(r,s) \in M} d(r, s)$

競合比～小さいほど望ましい評価指標～

ALG : $OMM(n)$ に対するオンラインアルゴリズム

ALG が α -競合的 とは $\forall \sigma: ALG(\sigma) \leq \alpha \cdot OPT(\sigma)$ が成り立つこと

$ALG(\sigma)$: σ を実行する際の ALG のコスト

$OPT(\sigma)$: σ 全体を既知としたときの最適コスト

$\mathcal{R}(ALG) = \inf \{ \alpha \mid ALG \text{ は } \alpha\text{-競合的} \} : ALG \text{ の競合比}$

≈ 最悪ケースにおいて理論上のコスト最小値の何倍以内に収まるか？

本日の内容

- OMMに対する古典的な結果を紹介
- OMMの拡張の一つであるオンライン輸送問題について最近の進展（自分の研究成果含む）について紹介

最初の結果

[Karyanasundaram and Pruhs, 1993] (以下[KP93]) は以下を示した :

1. $OMM(n)$ に対する貪欲法の競合比は $2^n - 1$,
2. Permutation アルゴリズムの競合比は $2n - 1$ でこれが最善.

つまり, 決定的アルゴリズムの競合比に関しては解決済み

※2については同時期に [Khuller et al., 1994] も独立に証明している

貪欲法 (*GREEDY*)

- まだリクエストがマッチしていないサーバの中で最も近いサーバに割り当てる

[KP93]は以下の2つを証明することにより, 貪欲法の $OMM(n)$ に対する競合比を $2^n - 1$ であると結論付けた.

定理 (競合比下界) ある $OMM(n)$ のインスタンスが存在して,

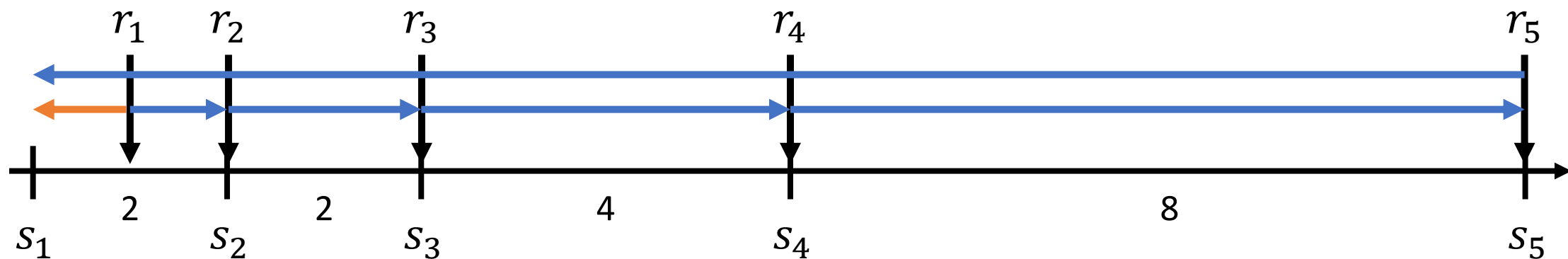
$$\frac{GREEDY(\sigma)}{OPT(\sigma)} = 2^n - 1$$

定理 (競合比上界) 任意の $OMM(n)$ のインスタンスに対して

$$\frac{GREEDY(\sigma)}{OPT(\sigma)} \leq 2^n - 1$$

競合比下界の証明

距離空間を直線とし, 下図のような例を考えればよい
(図は $n = 5$ の場合を示している)



$$GREEDY(\sigma) = 31 = 2^5 - 1, \quad OPT(\sigma) = 1$$

Permutation アルゴリズム

基本的なアイデア

オフライン最適割り当てと「**使用中のサーバの集合を一致**」させる
(ただしオンラインなので割り当ての中身は一致させられない)

M_i^* : 仮想的に内部で求める $r_1, \dots, r_i$ の最適割り当て

初期化: $M_0^* \leftarrow \emptyset$

For $i = 1, \dots, n$ do

 新しいリクエスト r_i を受け取る.

$r_1, \dots, r_i$ の最適割り当て M_i^* を求める.

M_i^* の中で M_{i-1}^* から新たに使用するようになったサーバを s_i とする.

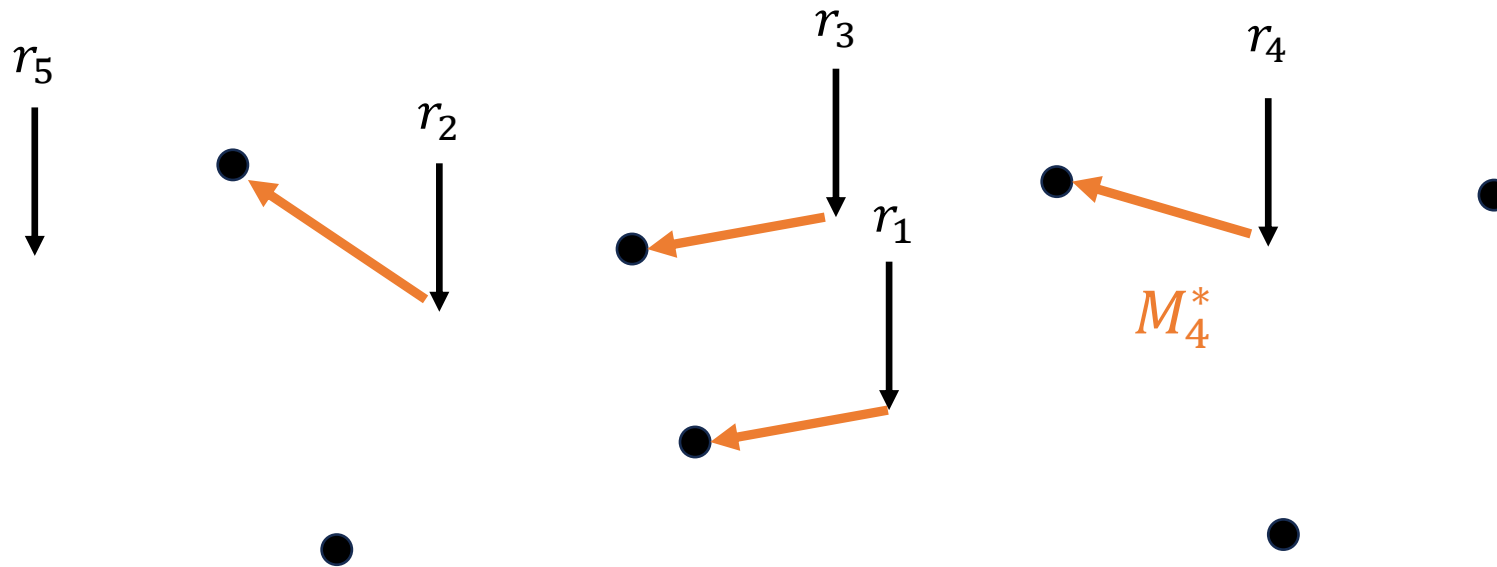
r_i を s_i に割り当てる.

end for

Permutation の割り当て 1

- サーバ（病院）
- ↓ リクエスト（患者）

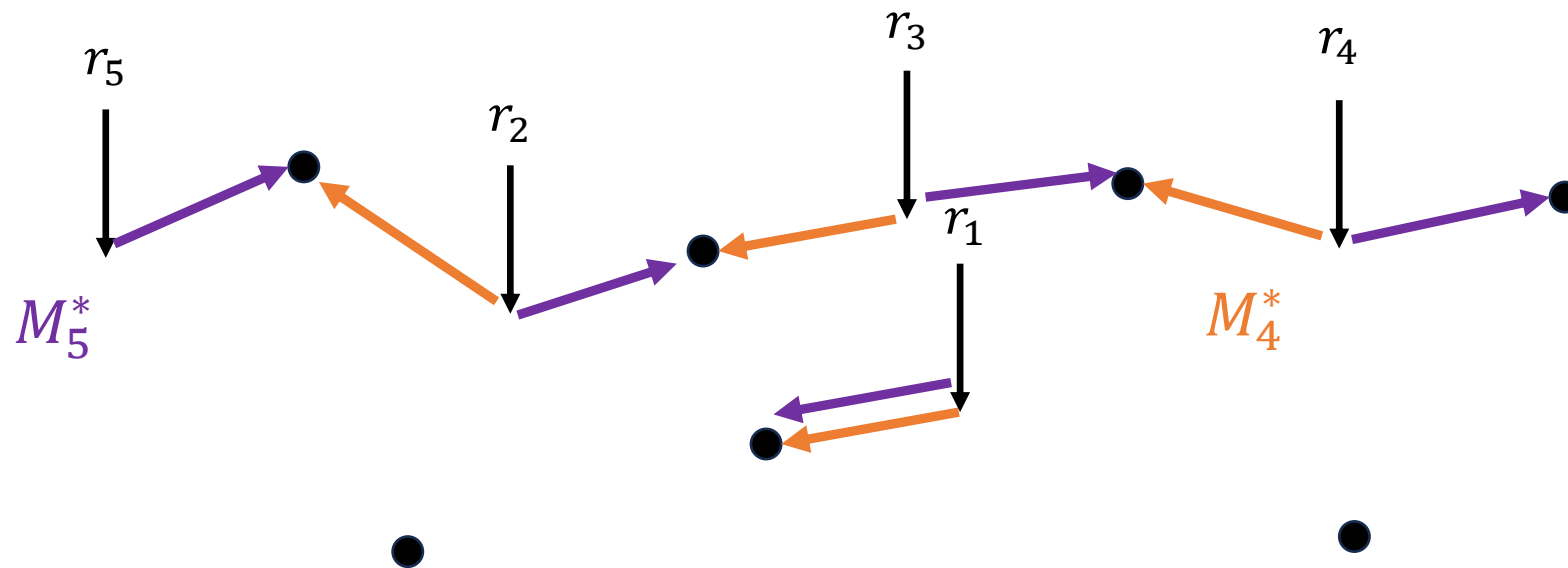
最適マッチング M_4^* を計算済みで新しいリクエスト r_5 を受け取った



Permutation の割り当て 2

- サーバ（病院）
- ↓ リクエスト（患者）

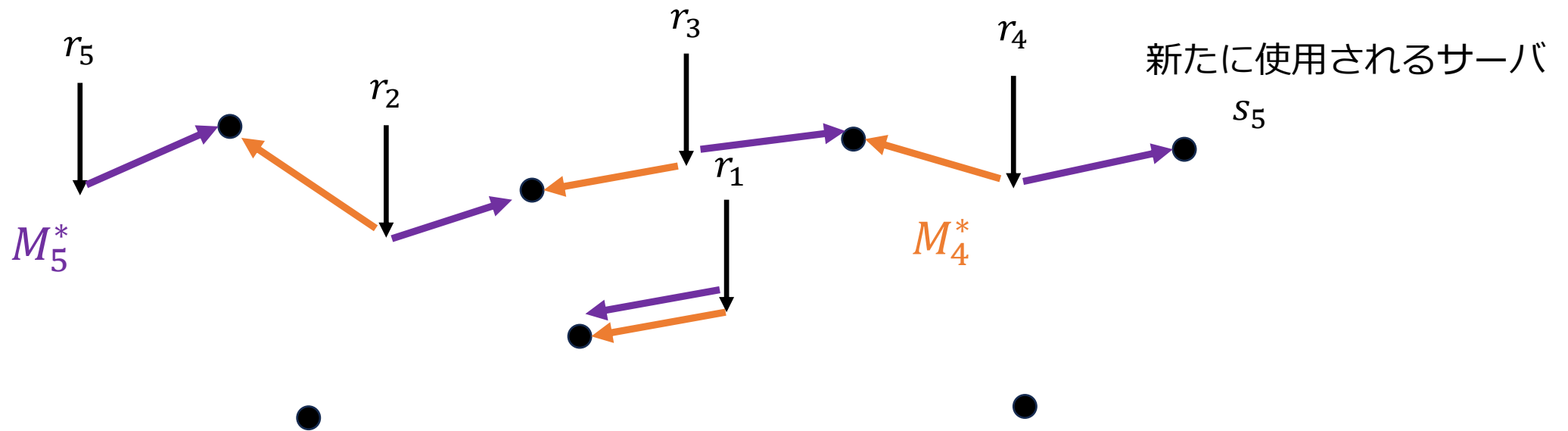
最適マッチング M_4^* を計算済みで新しいリクエスト r_5 を受け取った
 r_5 を含む最適マッチング M_5^* を計算する



Permutation の割り当て 3

- サーバ（病院）
- ↓ リクエスト（患者）

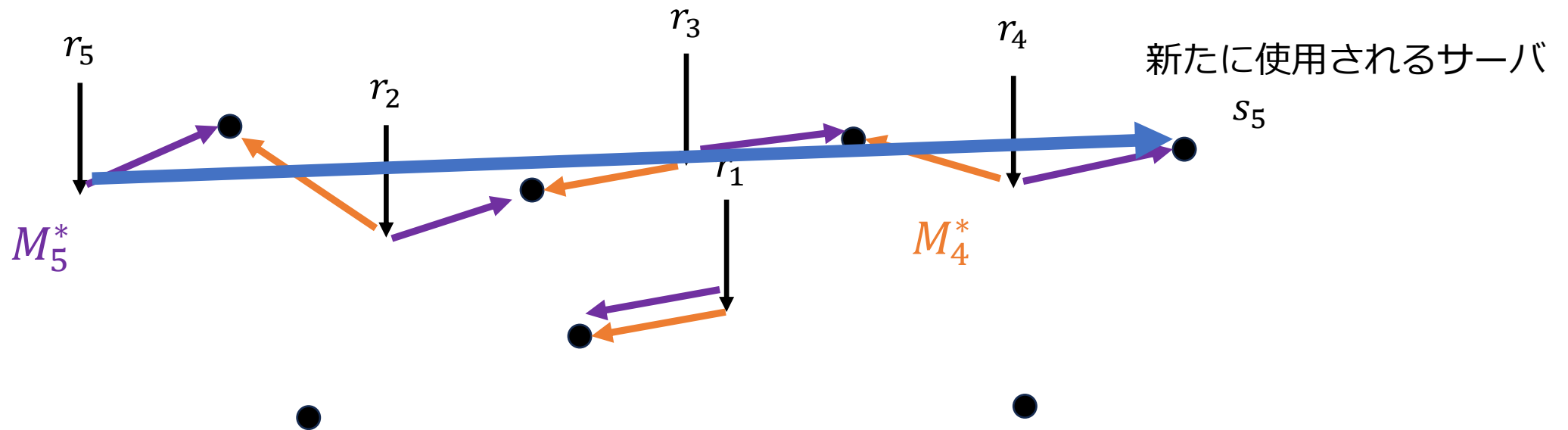
M_4^* と M_5^* を見比べて新たに使用されるサーバ s_5 を特定



Permutation の割り当て 4

- サーバ（病院）
- ↓ リクエスト（患者）

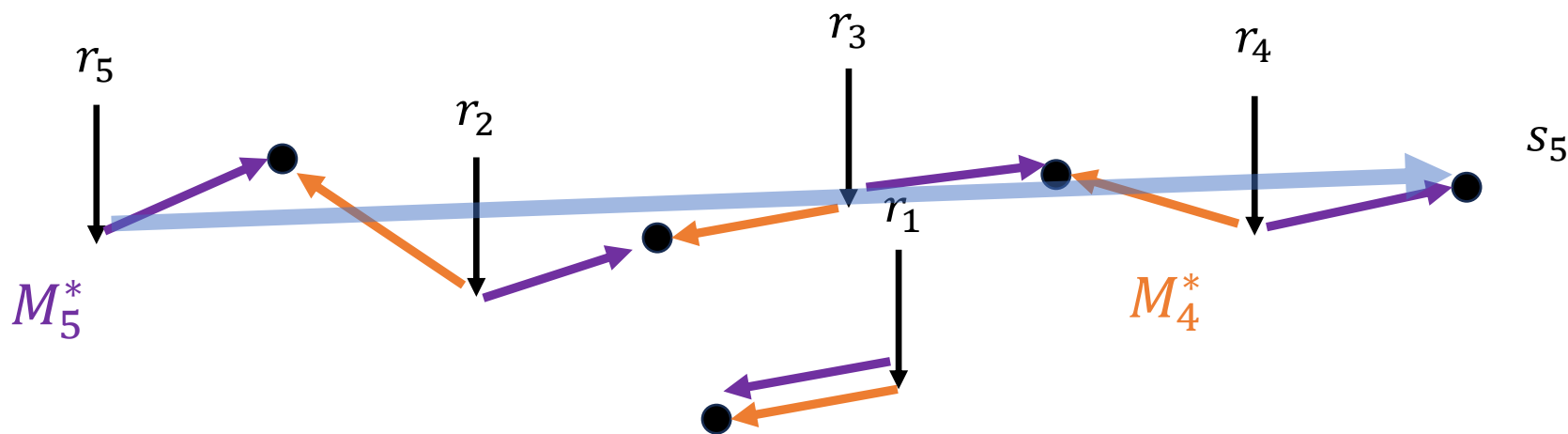
M_4^* と M_5^* を見比べて新たに使用されるサーバ s_5 を特定
 r_5 を s_5 に割り当て



Permutation の競合比解析 1

観察

- 三角不等式より r_5 の割当てコストは, M_4^* の重みと M_5^* の重みの和以下
 - 最適性より, M_4^* と M_5^* の重みは共に $OPT = M_n^*$ 以下
- よって (r_5 の割り当てコスト) $\leq 2 \cdot OPT$

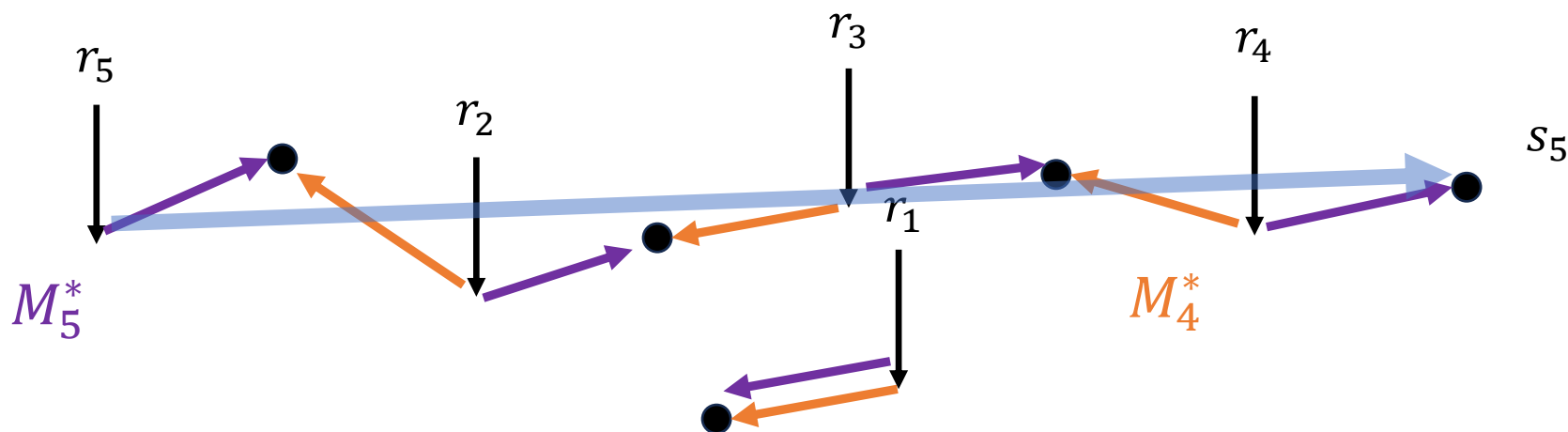


Permutation の競合比解析 2

同様に考えて各 $i = 1, \dots, n$ について (r_i の割り当てコスト) $\leq 2 \cdot OPT$
特に r_1 の割り当てコストは OPT 以下となるので,

$$Permutation \leq OPT + (n - 1) \cdot 2OPT = (2n - 1)OPT$$

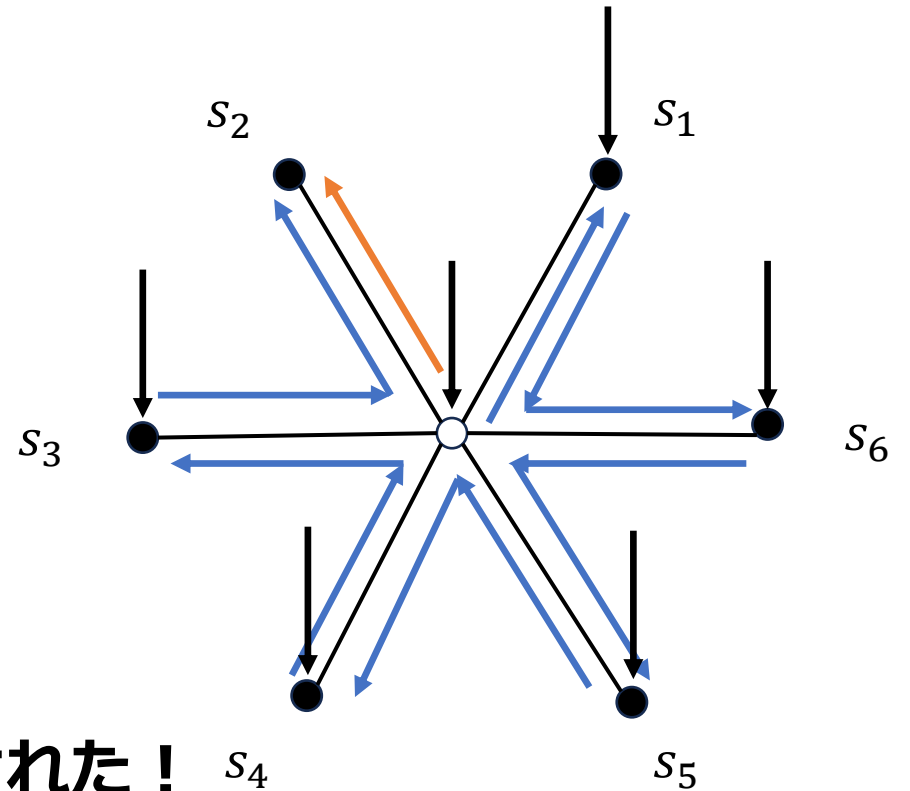
よって Permutation は $(2n - 1)$ -競合的であることが示された.



決定的アルゴリズムの競合比下界

任意の決定的アルゴリズム ALG の競合比が $2n - 1$ 以上であることを示す.

- Star グラフによる距離空間
- r_1 を中心の頂点に置く.
- 以降のリクエストは直前のリクエストの割り当て先に置く.
- 最適コストは 1 .
- ALG のコストは $1 + 2 \cdot (n - 1) = 2n - 1$.



Permutation が最善の競合比を持つことが示された！

OMMの拡張

乱択アルゴリズム 乱数を用い、最悪時ではなく平均的な競合比で評価する

パラメータ変更 競合比をサーバ数 n だけでないパラメータで表す

- [Nayyar and Raghvendra, 2017] による Robust-Matching アルゴリズムの研究
- オンライン輸送問題（容量付きOMM）

特殊な距離空間 特殊な距離空間上で更に競合比を改善する

- 直線
- Doubling Metric

リクエスト生成法 リクエストの生成法を変更する

- 既知の分布から i.i.d. に生成（Stochastic OMM）
- ランダム到着モデル
- 予測付きモデル

OMMの拡張

乱択アルゴリズム 乱数を用い、最悪時ではなく平均的な競合比で評価する

パラメータ変更 競合比をサーバ数 n だけでないパラメータで表す

- [Nayyar and Raghvendra, 2017] による Robust-Matching アルゴリズムの研究
- **オンライン輸送問題（容量付きOMM）**

特殊な距離空間 特殊な距離空間上で更に競合比を改善する

- 直線
- Doubling Metric

リクエスト生成法 リクエストの生成法を変更する

- 既知の分布から i.i.d. に生成（Stochastic OMM）
- ランダム到着モデル
- 予測付きモデル

オンライン輸送問題（容量付きOMM）

オンライン輸送問題（Online Transportation, Online Metric b-Matching, etc.）

サーバが距離空間上の m 箇所に配置され、各サーバ s_i は**容量** c_i を持つ
1つのサーバに c_i 個までのリクエストを割り当て可能

※OMMを特殊ケースとして含む（容量がすべて1の場合）

「容量 c_i の1つのサーバ」を「容量1のサーバが同一地点に c_i 個」
と捉えることでサーバ数が $n = \sum_{i=1}^m c_i$ である OMM と捉えることが可能

しかし、総容量 $n = \sum_{i=1}^m c_i$ でなく**サーバ数 m に関する競合比**を得たい！

先行研究と課題

m : サーバ数
 n : 総容量
※一般に $m \ll n$

- 貪欲法の競合比は $2^m - 1$ [Kalyanasundaram and Pruhs, 2000], [Harada et al., 2024]
- Permutation の競合比は $\Omega(n)$ [Kalyanasundaram and Pruhs, 2000], [Nicoletti, 2024]
- 自明な競合比下界 $2m - 1$

OMMに対する有名な二つのアルゴリズムはどちらも今一つ

- OMMに対して最善の Permutation は容量が増えると競合比が悪化
- 貪欲法はサーバ数のみに依存する競合比を持つが、下界とのギャップ大

予想 [Kalyanasundaram and Pruhs, 1998] オンライン輸送問題に対して競合比が $2m - 1$ 以下となる決定的アルゴリズムは存在するか？

研究成果

予想 [Kalyanasundaram and Pruhs, 1998] オンライン輸送問題に対して競合比が $2m - 1$ 以下となる決定的アルゴリズムは存在するか？

Robust-Matching が達成した $O(m \log^2 n)$ [Nayyar and Raghvendra, 2017] 以来進展がなかったが...

定理 [Harada and Itoh, 2025]

競合比が $8m - 5$ 以下となる決定的アルゴリズムが存在する

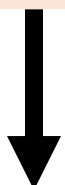
✓ 下界 $2m - 1$ に初めて定数倍の範囲で近づき、オーダーの意味で最善 [Harada and Itoh, 2025] はアルゴリズムは単純だが、解析が複雑だった。 [Arndt et al., 2026] が解析を大幅に単純化した。

(後日談, 未発表) 実は Robust-Matching も $O(m)$ -競合的な可能性が高い

アルゴリズム設計の流れ

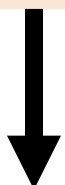
一般の距離空間をより単純な「木距離」に対する問題に帰着させる

STEP 1 : **重み無し木**に対する単純な**深さ優先探索**アルゴリズム



重みの階層ごとに再帰的に使用

STEP 2 : **重みが2の冪**であるような木に対するアルゴリズム



最小全域木による埋め込み + 重みの丸め

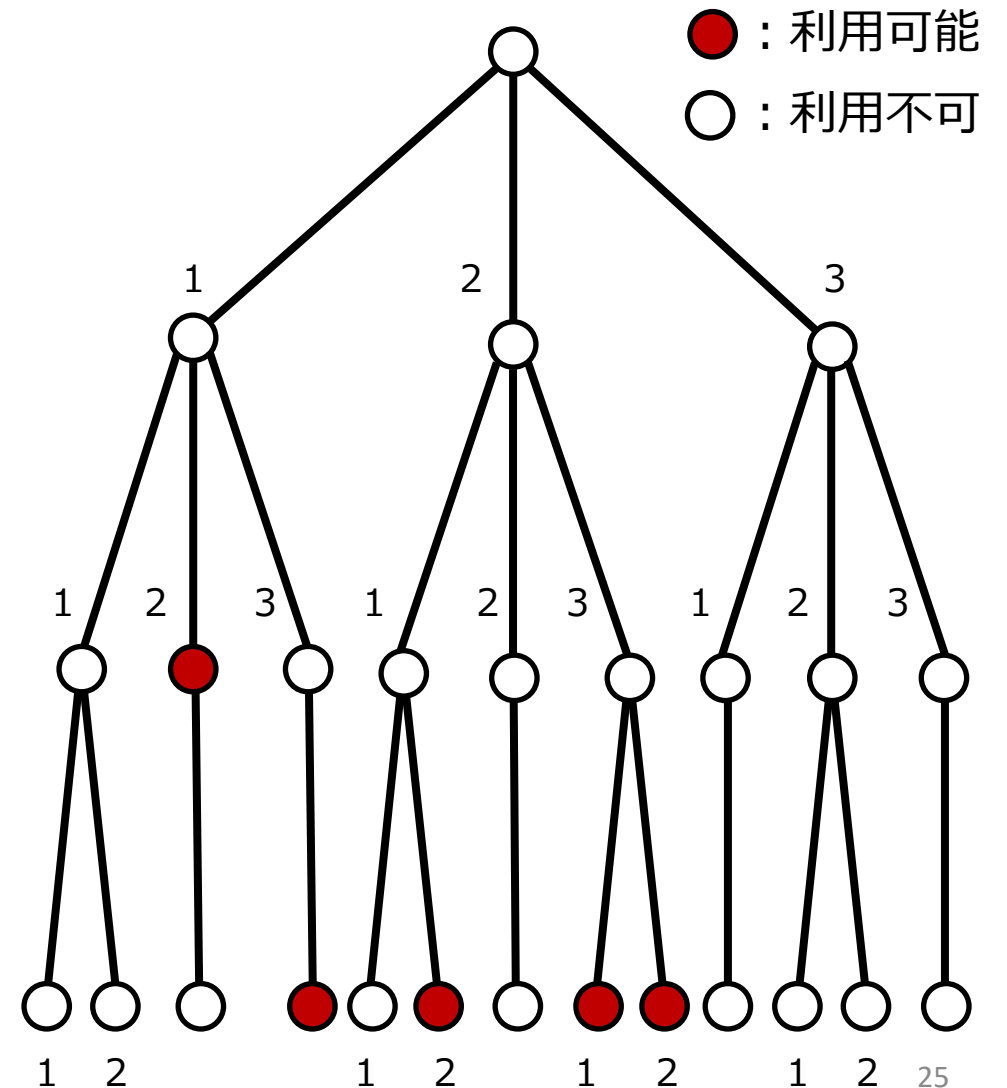
STEP 3 : **一般の距離空間**に対するアルゴリズム

重み無し木に対するアルゴリズム 1

- 任意の頂点を根として選ぶ
- 2人以上子を持つ頂点に対して, 子に任意に番号を振る

新しいリクエストが頂点 v に生起:

v から**深さ優先探索**を行い, 初めて見つかった利用可能なサーバに割り当てる



重み無し木に対するアルゴリズム 2

- 任意の頂点を根として選ぶ
- 2人以上子を持つ頂点に対して, 子に任意に番号を振る

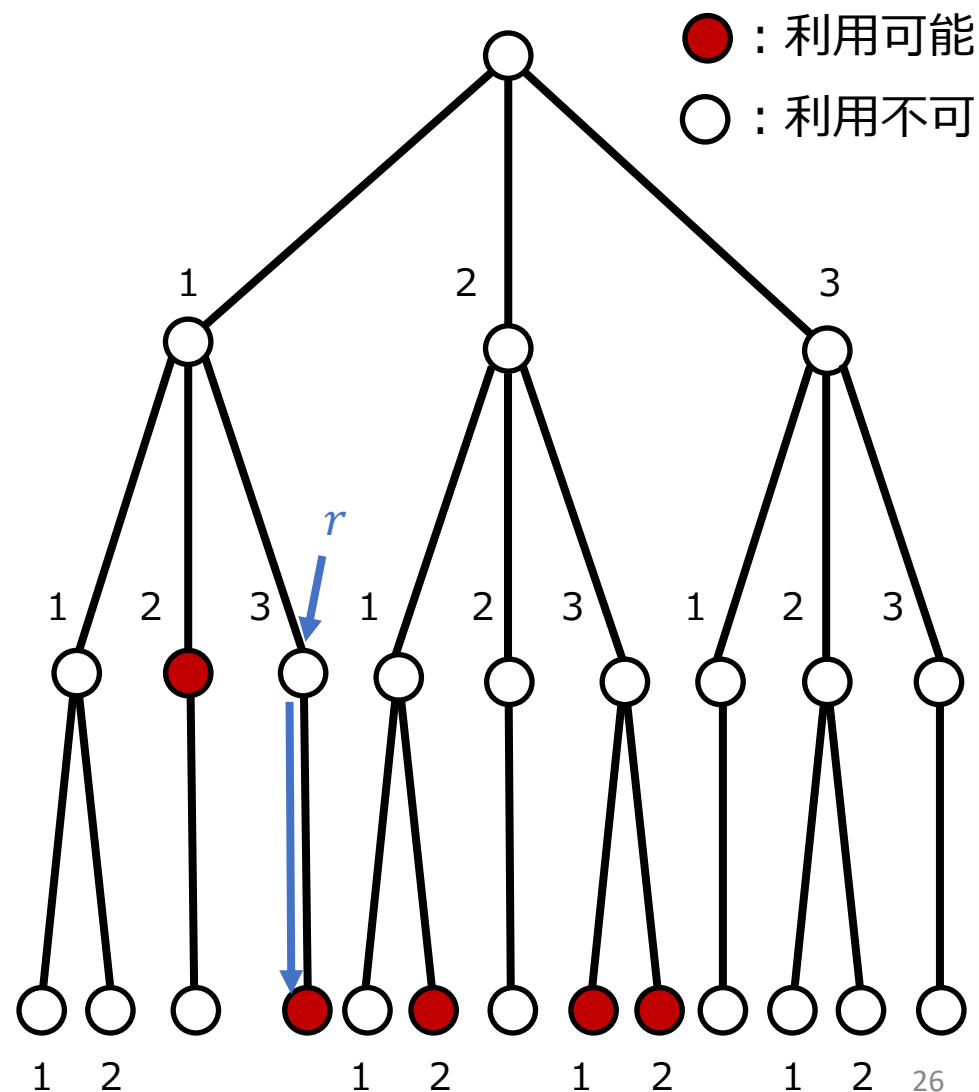
新しいリクエストが頂点 v に生起:

IF $\exists v$ の子孫かつ利用可能なサーバ, **THEN**

そのサーバに割り当てる. 複数のサーバが利用可能なときは, 番号の若い子を辿る.

ELSE

子孫に利用可能なサーバが現れるまで親を辿ってから, 上記と同じことをする.



重み無し木に対するアルゴリズム 3

- 任意の頂点を根として選ぶ
- 2人以上子を持つ頂点に対して, 子に任意に番号を振る

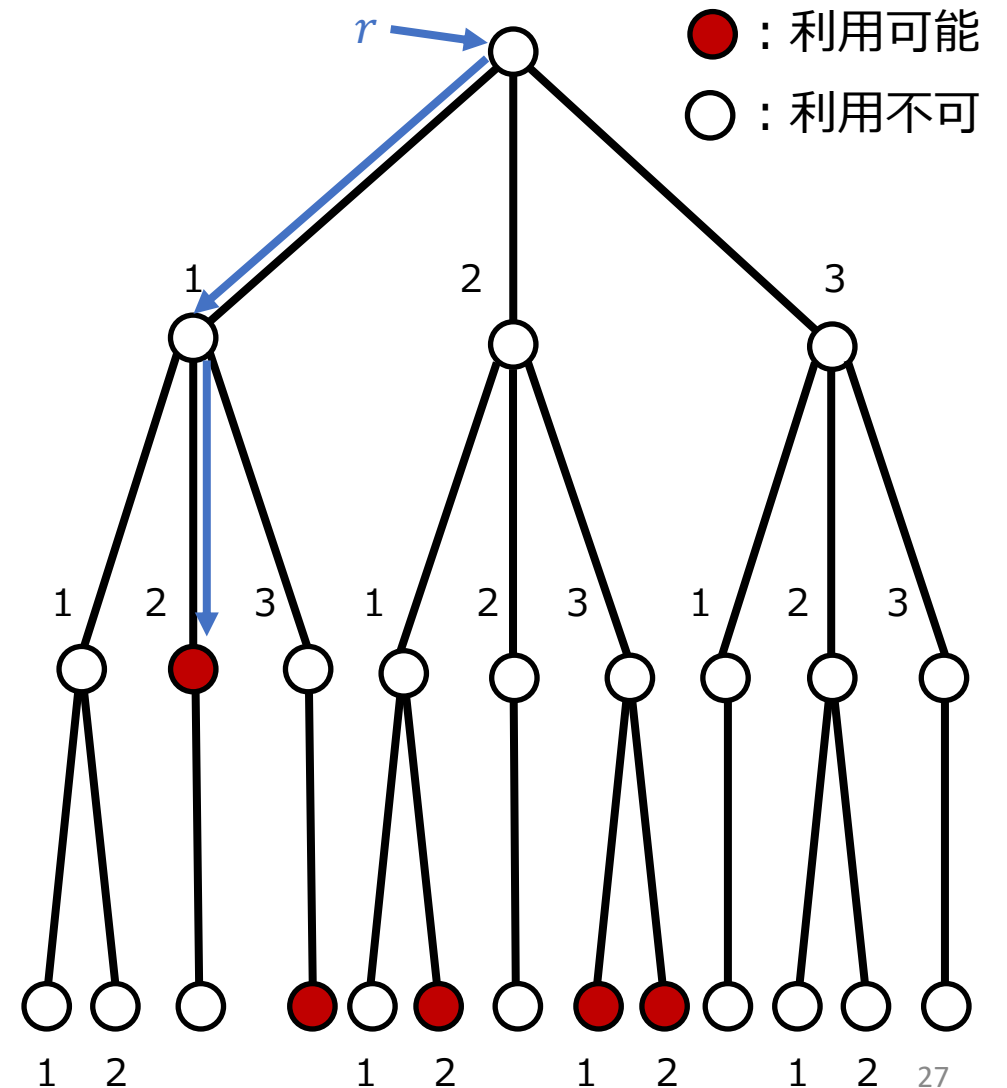
新しいリクエストが頂点 v に生起:

IF $\exists v$ の子孫かつ利用可能なサーバ, **THEN**

そのサーバに割り当てる. 複数のサーバが利用可能なときは, 番号の若い子を辿る.

ELSE

子孫に利用可能なサーバが現れるまで親を辿ってから, 上記と同じことをする.



2の冪乗の重み付き木に拡張

- 任意の頂点を根として選ぶ
- 2人以上子を持つ頂点に対して, 子に任意に番号を振る
- 木を**重み最大の辺を含まない**極大連結成分に分割

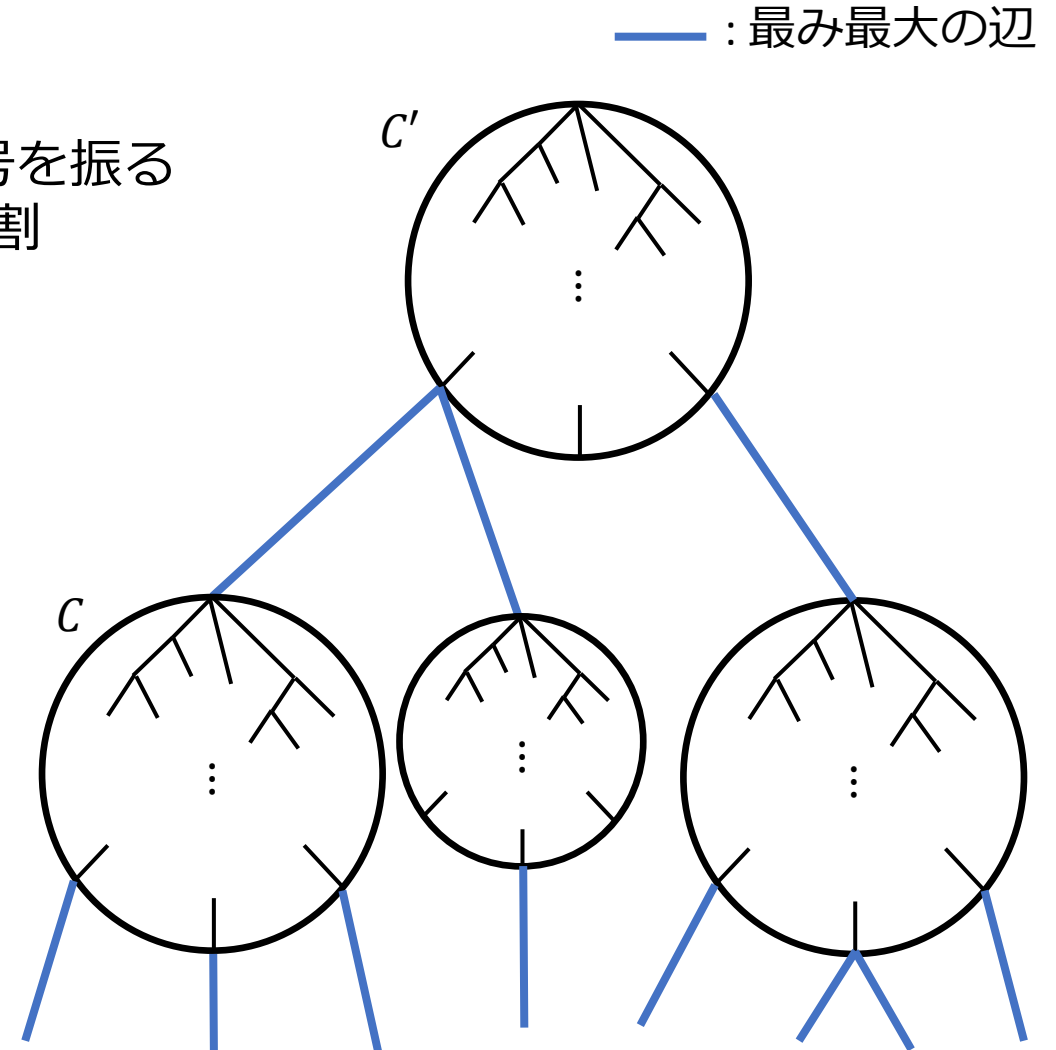
新しいリクエストが連結成分 C 上に生起:

IF $\exists C$ 内に利用可能なサーバ, **THEN**

C に対して帰納的に定義されたアルゴリズムに従って割り当てを行う.

ELSE

重み無し木に対するアルゴリズムを用いて, 利用可能なサーバを含む別の連結成分 C' を選び, C' に対して帰納的に定義されたアルゴリズムに従って割り当てを行う.



※Base case: サーバが1つの場合, リクエストをすべて唯一のサーバに割り当てる

一般の距離空間への拡張

1. 与えられた m 点距離空間を重み付き m 頂点完全グラフとして見たときの最小全域木 T を求める
2. 最小全域木 T の各辺の重みを2の冪に切り上げ (T' とする)
3. 重み付き T' に対するアルゴリズムに従って割り当て



2の冪への切り上げ: $2^{i-1} < w \leq 2^i$ のとき重み w を 2^i に変更する

まとめ

- OMMについて

- 素朴な貪欲法の競合比は $2^n - 1$ であまり良くない
- Permutation が最善の競合比 $2n - 1$ を達成
- 「使用中のサーバ集合」をOPTと揃えることが重要

- OMMには様々な拡張がある

- オンライン輸送問題について

- サーバが容量を持つ
- 総容量ではなくサーバ数に関する競合比を考える
- $O(m)$ -競合的なアルゴリズムが存在
- Permutationや貪欲法では上手くいかないが木での単純なDFSアルゴリズムを拡張すると上手くいく

以下補足

これ以降は補足スライド

Permutation の準備：増加道法

付け替えが許される場合に，逐次的に最適割り当てを求める手法

$r_1, \dots, r_{i-1}$ の最適マッチング M_{i-1}^* が求まっているとするとき，
 $r_1, \dots, r_{i-1}, r_i$ の最適マッチング M_i^* は次のように求められる：

r_i から始まって M_{i-1}^* に含まれる辺とそうでない辺を交互に通りながら未使用のサーバ s_i で終わるパス（増加道） P_i の中で

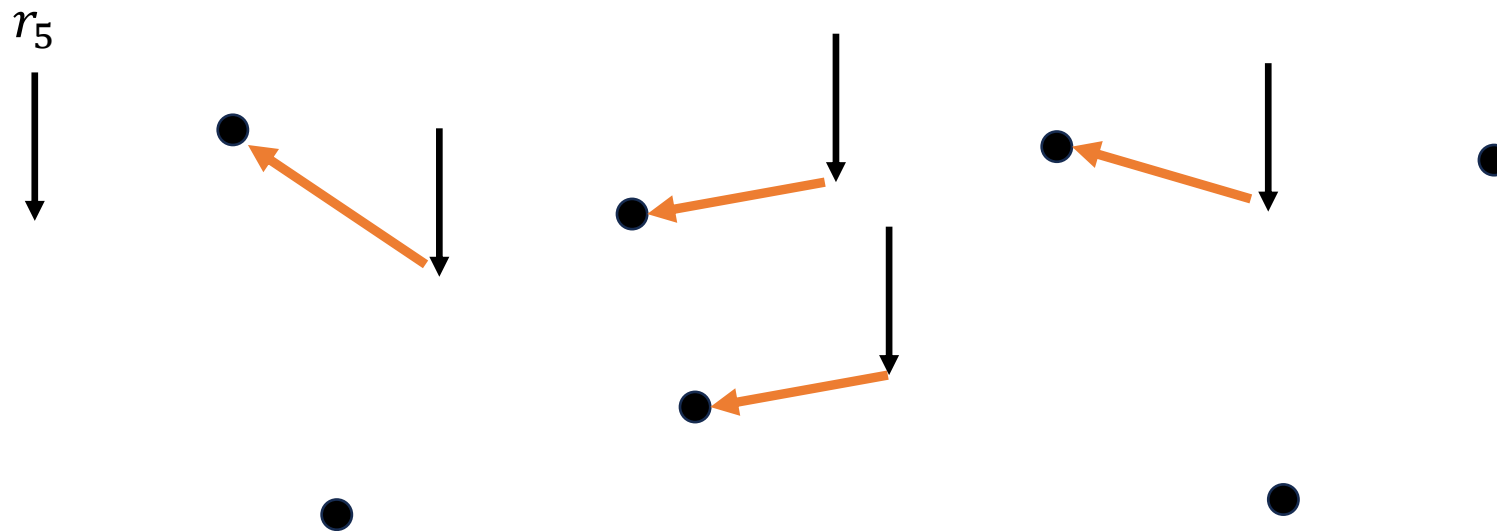
$$\phi(P_i) := \sum_{(r,s) \in P_i \setminus M_{i-1}^*} d(r,s) - \sum_{(r,s) \in P_i \cap M_{i-1}^*} d(r,s)$$

を最小化するものを求め， $M_i^* := M_{i-1}^* \oplus P_i$ とする

対称差： $A \oplus B := (A \setminus B) \cup (B \setminus A)$

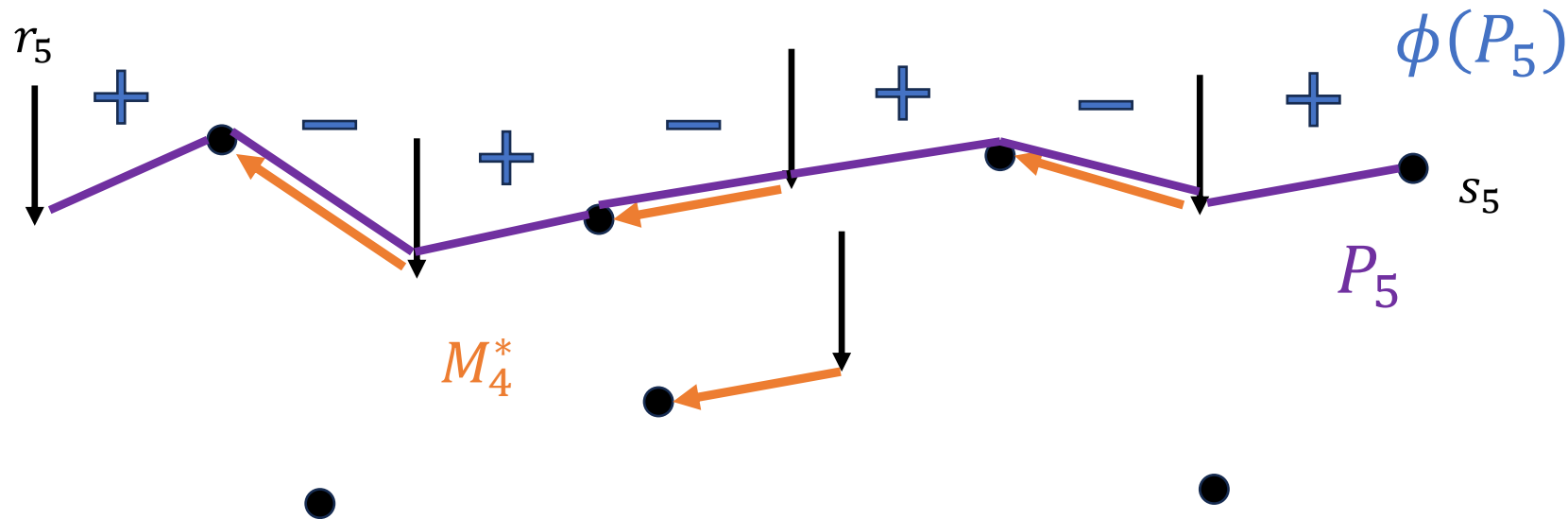
増加道法のイメージ 1

- 最適マッチング M_4^* が与えられて、新しいリクエスト r_5 を受け取った



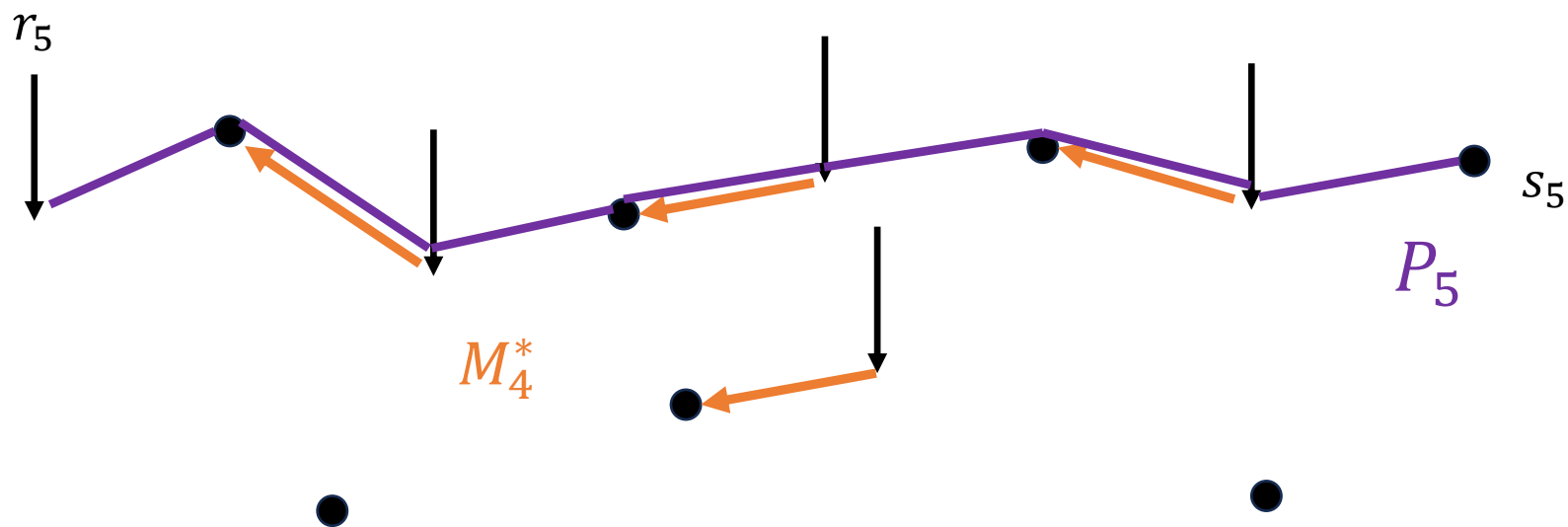
増加道法のイメージ 2

- 最適マッチング M_4^* が与えられて、新しいリクエスト r_5 を受け取った
- ϕ を最小化する増加道 P_5 を求める



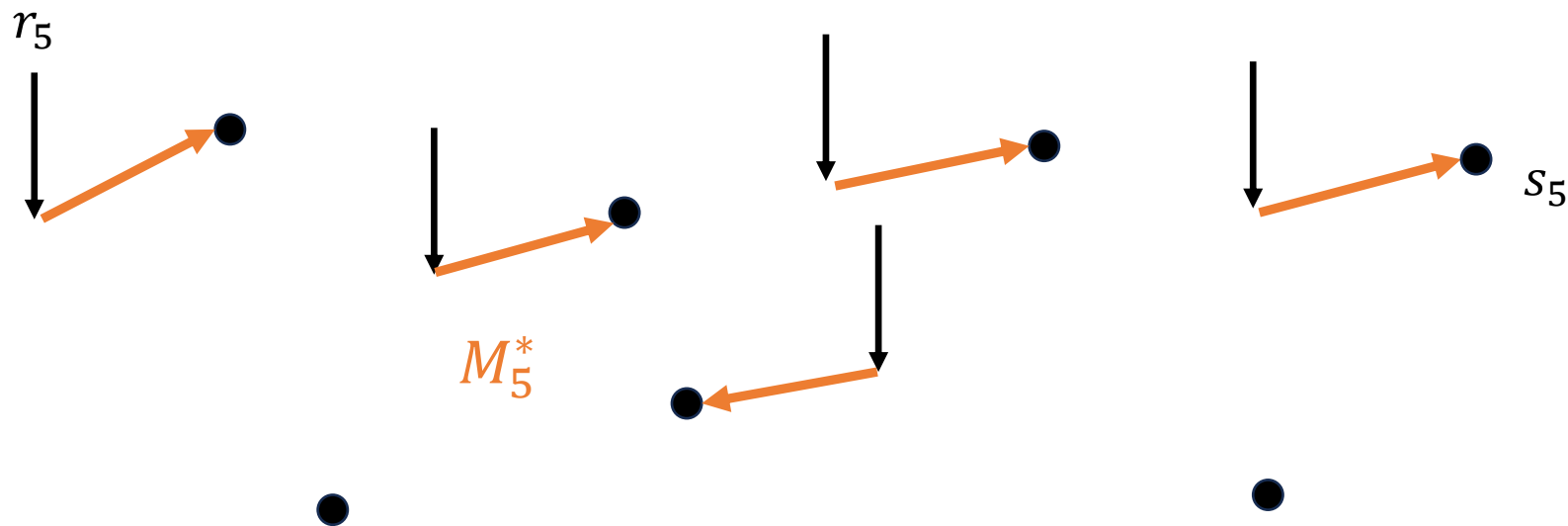
増加道法のイメージ 3

- 最適マッチング M_4^* が与えられて、新しいリクエスト r_5 を受け取った
- ϕ を最小化する増加道 P_5 を求める
- P_5 に従って、 M_4^* を $M_5^* := M_4^* \oplus P_5$ に更新する



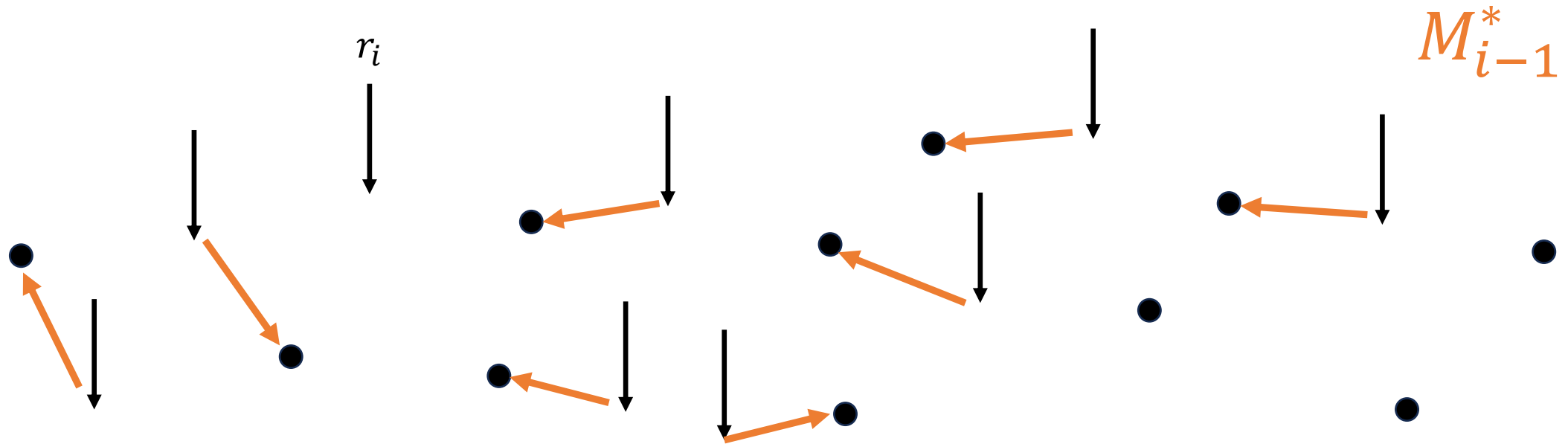
増加道法のイメージ 3

- 最適マッチング M_4^* が与えられて、新しいリクエスト r_5 を受け取った
- ϕ を最小化する増加道 P_5 を求める
- P_5 に従って、 M_4^* を $M_5^* := M_4^* \oplus P_5$ に更新する



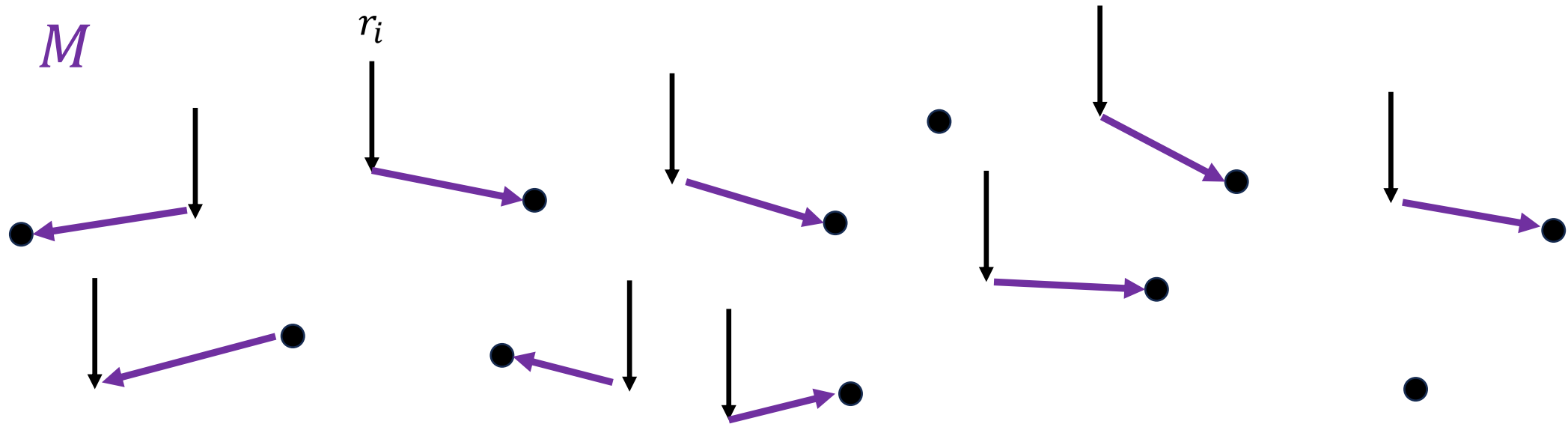
最適性の説明 1

- $r_1, \dots, r_{i-1}$ の最適マッチング M_{i-1}^* を固定
- 任意に $r_1, \dots, r_{i-1}, r_i$ の最適マッチング M を取り, $M_{i-1}^* \oplus M$ を取る
- $M_i^* \oplus M$ は r_i を端点に持つ増加道 P が存在する



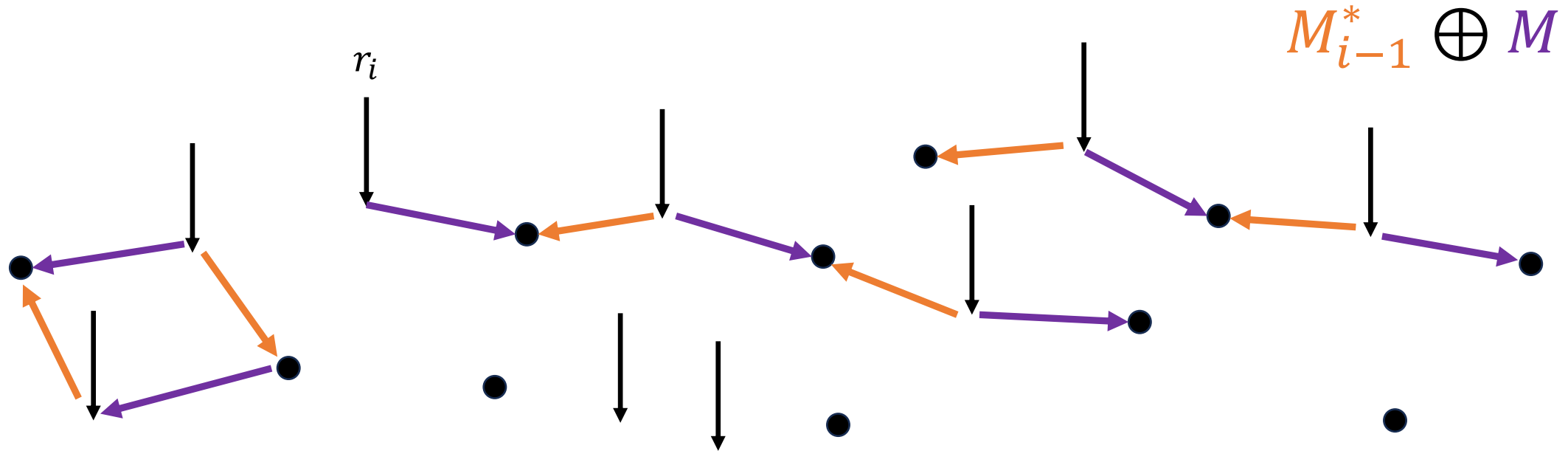
最適性の説明 1

- $r_1, \dots, r_{i-1}$ の最適マッチング M_{i-1}^* を固定
- 任意に $r_1, \dots, r_{i-1}, r_i$ の最適マッチング M を取り, $M_{i-1}^* \oplus M$ を取る
- $M_i^* \oplus M$ は r_i を端点に持つ増加道 P が存在する



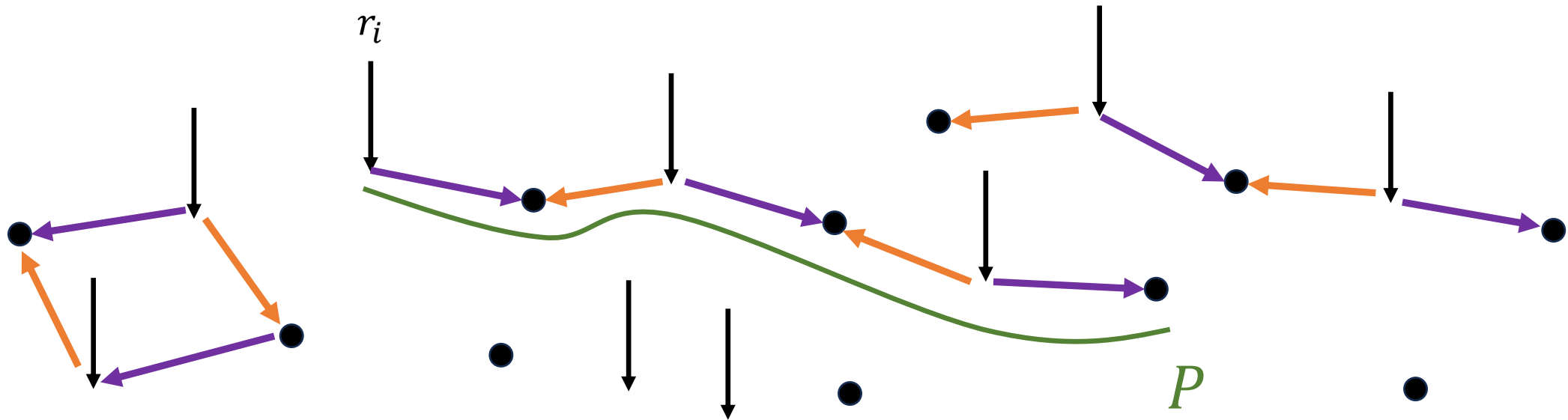
最適性の説明 1

- $r_1, \dots, r_{i-1}$ の最適マッチング M_{i-1}^* を固定
- 任意に $r_1, \dots, r_{i-1}, r_i$ の最適マッチング M を取り, $M_{i-1}^* \oplus M$ を取る
- $M_i^* \oplus M$ は r_i を端点に持つ増加道 P が存在する



最適性の説明 1

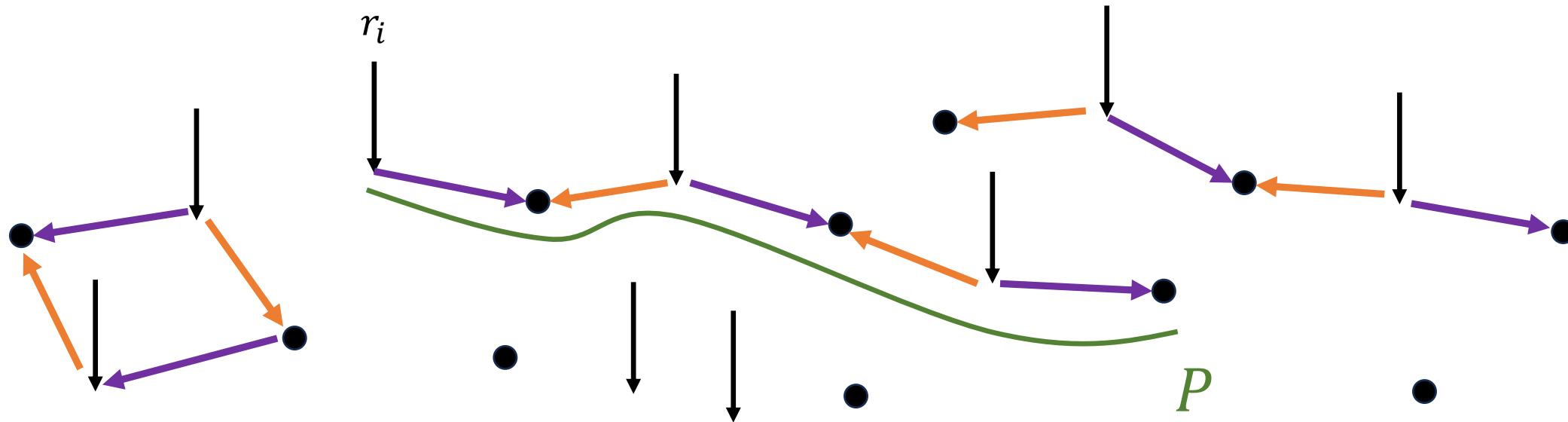
- $r_1, \dots, r_{i-1}$ の最適マッチング M_{i-1}^* を固定
- 任意に $r_1, \dots, r_{i-1}, r_i$ の最適マッチング M を取り, $M_{i-1}^* \oplus M$ を取る
- $M_i^* \oplus M$ は r_i を端点に持つ増加道 P が存在する



最適性の説明 2

$$d(M) := \sum_{(r,s) \in M} d(r,s)$$

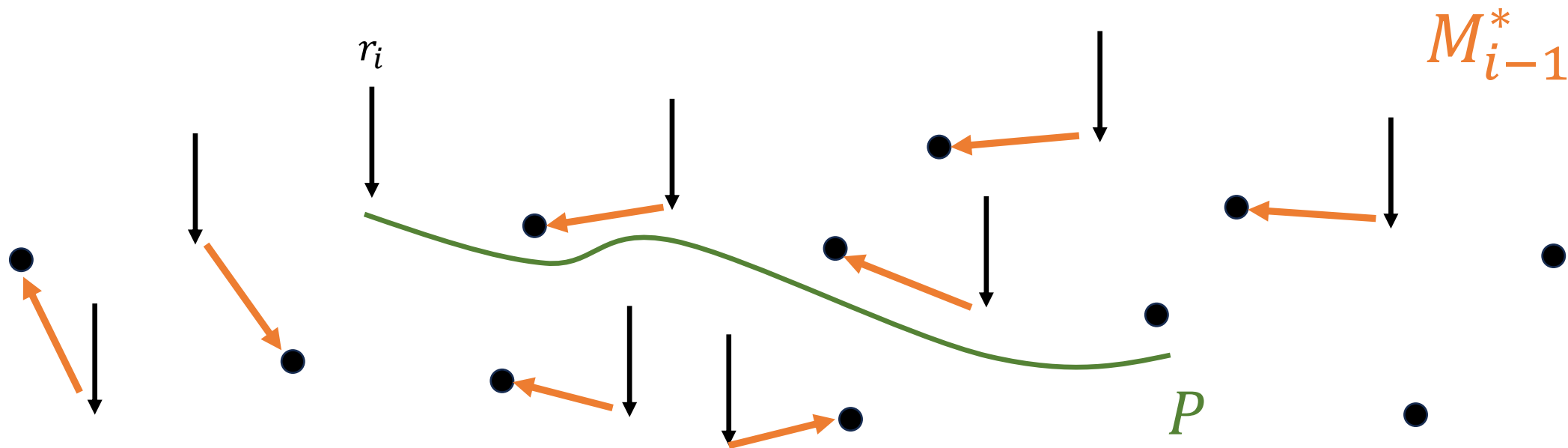
- $M_{i-1}^* \oplus M$ の P 以外の部分でかかるコストが異なると最適性に矛盾
- よって $M_{i-1}^* \oplus P$ もまた $r_1, \dots, r_i$ の最適マッチングとなる
- $\phi(P)$ は $d(M_{i-1}^* \oplus P) - d(M_{i-1}^*)$ を意味するのでこれを最小化すれば良い



最適性の説明 2

$$d(M) := \sum_{(r,s) \in M} d(r,s)$$

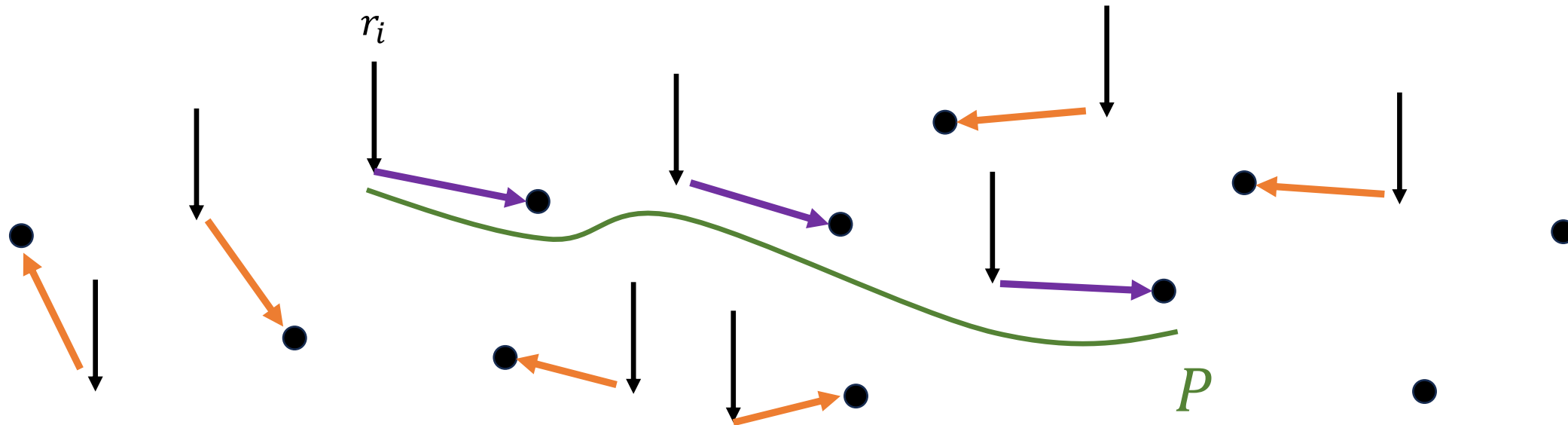
- $M_{i-1}^* \oplus M$ の P 以外の部分でかかるコストが異なると最適性に矛盾
- よって $M_{i-1}^* \oplus P$ もまた $r_1, \dots, r_i$ の最適マッチングとなる
- $\phi(P)$ は $d(M_{i-1}^* \oplus P) - d(M_{i-1}^*)$ を意味するのでこれを最小化すれば良い



最適性の説明 2

$$d(M) := \sum_{(r,s) \in M} d(r,s)$$

- $M_{i-1}^* \oplus M$ の P 以外の部分でかかるコストが異なると最適性に矛盾
- よって $M_{i-1}^* \oplus P$ もまた $r_1, \dots, r_i$ の最適マッチングとなる
- $\phi(P)$ は $d(M_{i-1}^* \oplus P) - d(M_{i-1}^*)$ を意味するのでこれを最小化すれば良い



Permutation アルゴリズム

M_i : アルゴリズムが実際に出力する $r_1, \dots, r_i$ の割り当て

M_i^* : アルゴリズムが仮想的に内部で求める $r_1, \dots, r_i$ の最適割り当て

初期化: $M_0 \leftarrow \emptyset, M_0^* \leftarrow \emptyset$

For $i = 1, \dots, n$ do

新しいリクエスト r_i を受け取る.

ϕ を最小化するような M_{i-1}^* に対する増加道 P_i を求める.

P_i の r_i でない方の端点 (未使用サーバ) を s_i とする.

更新: $M_i \leftarrow M_{i-1} \cup \{(r_i, s_i)\}, M_i^* \leftarrow M_{i-1}^* \oplus P_i$

end for

Output: M_n

Robust-Matching アルゴリズム (RM)

Permutation アルゴリズムの一般化 [Raghvendra, 2016]

ϕ の代わりにパラメータ $t \geq 1$ を用いて ϕ_t を最小化する増加道 P_i を用いる

$$\phi_t(P_i) := t \cdot \sum_{(r,s) \in P_i \setminus M_{i-1}^*} d(r,s) - \sum_{(r,s) \in P_i \cap M_{i-1}^*} d(r,s)$$

特徴

- $t = 1$ なら Permutation に一致 (競合比 $2n - 1$)
- (t を定数扱いするなら) 任意の t に対して競合比 $O(n)$
- $t \rightarrow \infty$ のときも最善の競合比 $2n - 1$ に漸近
- 他の様々な設定でも柔軟に最適性を達成 (一部は後述)

RMのパラメータの直感的意味

$$\phi_t(P_i) := t \cdot \sum_{(r,s) \in P_i \setminus M_{i-1}^*} d(r,s) - \sum_{(r,s) \in P_i \cap M_{i-1}^*} d(r,s)$$

仮想的割り当て M_i^* を求める際に以下をバランスする

1. 「付け替え」をどれくらい嫌うか

t が大きいほど辺の本数が少ない増加道を選びやすくなる

2. そのトレードオフとして最適性をどのくらい犠牲にするか

M_i^* は Permutation の場合は $r_1, \dots, r_i$ の最適割り当てだったが, RMでは最適割り当てコストの t 倍以下という保証になる

決定的アルゴリズムの結論と課題

結論

- $OMM(n)$ に対するアルゴリズムの競合比は最善でも $2n - 1$ であり, それは増加道法に基づく Permutation アルゴリズムによって達成される.
- 一見自然に見える貪欲法の競合比は $2^n - 1$ となり, 競合比という観点から見ると性能は劣る.

課題

- 競合比 $2n - 1$ という結論が悲観的すぎる.

原因: 任意に悪意のあるリクエスト列を作成できるという仮定が強すぎる.
もう少しアルゴリズム側にとって有利な (= 現実的な) 設定を考えたい.

乱択化

乱数を用いて確率的にマッチングを出力することを考える.

リクエスト列の作成者：アルゴリズムは既知，乱数の実現値は**未知**.

アルゴリズムの**平均コスト**を下げることを目的にする.

Oblivious Adversary

乱択アルゴリズム ALG に対する競合比の定義は下式に置き換える：

$$\mathbb{E}[ALG(\sigma)] \leq \alpha \cdot OPT(\sigma)$$

上式が成り立つとき ALG を α -競合的といい， α -競合的となるような α の下限値を ALG の競合比という.

乱択アルゴリズムの競合比

競合比上界

Randomized Greedy [Meyerson et al., 2006]

- 一様距離空間において競合比 $O(\log n)$
- 一般の距離空間では $O(\log^3 n)$

[Bansal et al., 2007]

- 競合比 $O(\log^2 n)$ に改善

競合比下界

- 任意の乱択アルゴリズムの競合比は $\Omega(\log n)$ [Meyerson et al., 2006]

扱う範囲

- 一様距離空間における解析
- [Meyerson et al., 2006], [Bansal et al., 2007] の提案した解析上のテクニック

$O(\log^2 n)$ -競合的なアルゴリズムの具体的な解析までは踏み込まない

競合比下界

一様距離空間上で考える

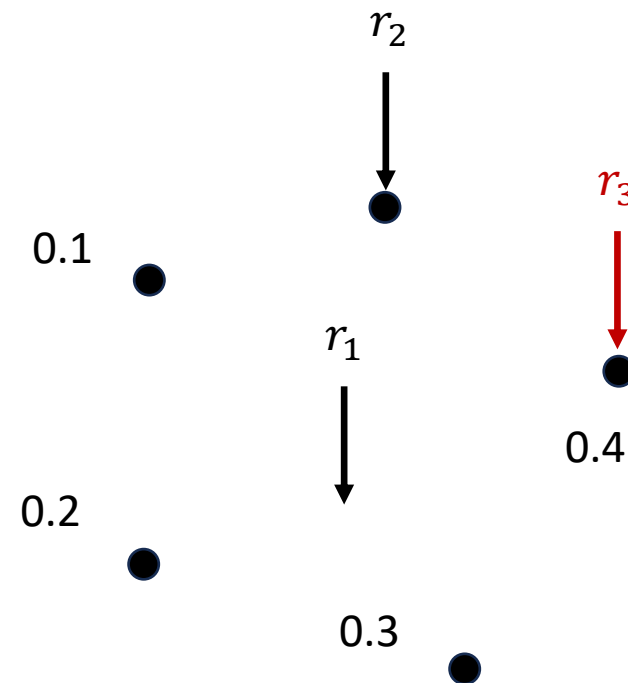
コスト 0 でリクエストの割り当てができるなら必ずするアルゴリズムしか考える必要はない

リクエストは次のように置く

- 1つ目はサーバの無い場所
- 2つ目は最も使用不可である確率の高いサーバの上
- 以降はまだリクエストを置いていないサーバの中で最も使用不可である確率の高い場所

$i \geq 2$ 番目のリクエストの期待コスト $= \frac{1}{n-i+1}$

$$\mathbb{E}[ALG] = 1 + \frac{1}{n} + \frac{1}{n-1} + \cdots + \frac{1}{2} = H_n = \Omega(\log n), \quad OPT = 1$$



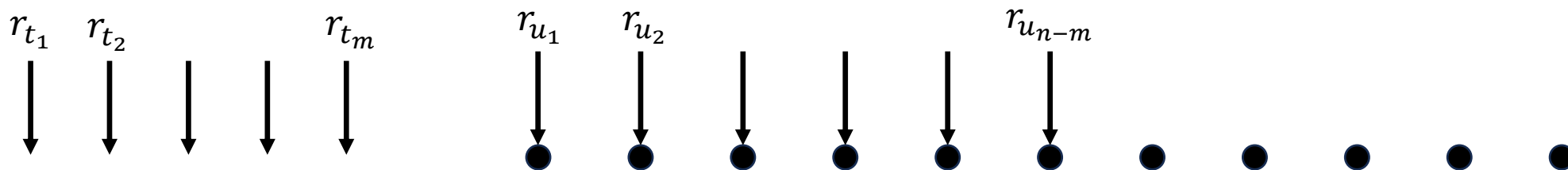
一様距離空間上の競合比上界 1

アルゴリズム *ALG*

- コスト 0 での割り当てが可能であれば必ずする
 - そうでなければ空いているサーバから一様ランダムに選んで割り当てる
- このアルゴリズムが $O(\log n)$ 競合的となることを示す.

任意に $\sigma = r_1 \cdots r_n$ を固定する

OPT がコスト 1 を払うようなリクエストはサーバ上に無いとしてよい (あったとしてもサーバの無い点に移動すれば *OPT* は不変, *ALG* は減らない)



一様距離空間上の競合比上界 2

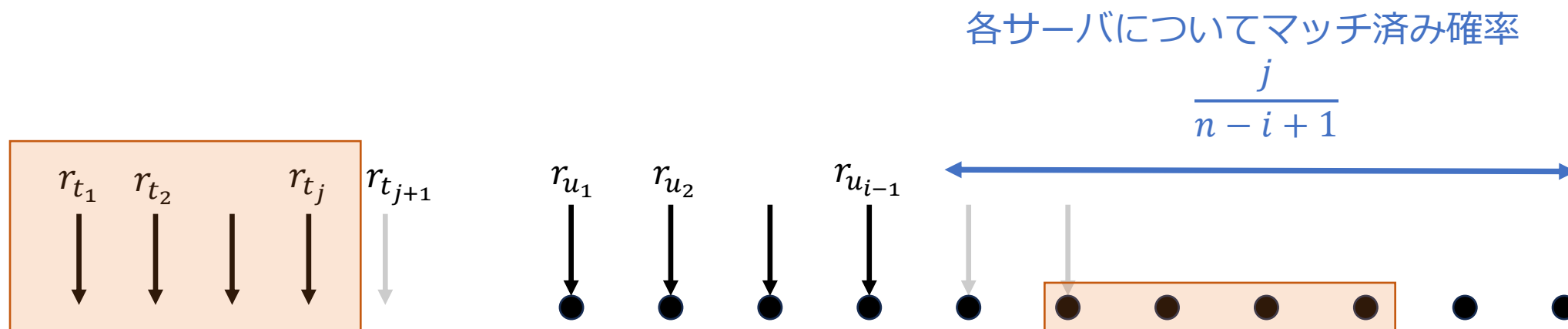
リクエスト r_{u_i} の割り当て直前に注目する ($t_j < u_i < t_{j+1}$)

サーバ上には $i - 1$ 個のリクエスト

サーバの無い場所に j 個のリクエストがある

まだリクエストがその上に無い $n - i + 1$ 個のサーバの内 j 個は利用不可能

リクエストが置かれていないサーバはすべて対等



一様距離空間上の競合比上界 3

リクエスト r_{u_i} の割り当てにかかる期待コスト

= r_{u_i} の場所がマッチ済みである確率 $= \frac{j}{n-i+1} \leq \frac{m}{n-i+1}$

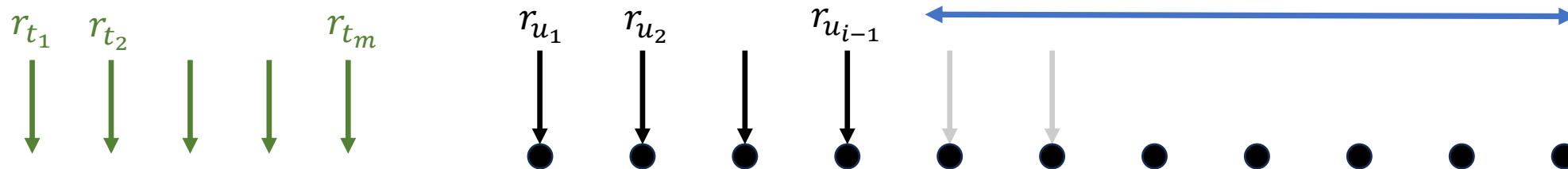
よって

$$H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$$

$$\mathbb{E}[ALG] \leq m + \sum_{i=1}^{n-m} \frac{m}{n-i+1} \leq \sum_{i=1}^n \frac{m}{n-i+1} = m \cdot H_n = H_n \cdot OPT$$

各サーバについてマッチ済み確率

$$\frac{j}{n-i+1}$$



距離空間の制限

リクエストの到着をサーバのある n 点のみに制限してアルゴリズムを設計しても競合比のオーダーには影響がない

補題 [Meyerson et al., 2006] すべてのリクエストがサーバ上にあるような任意のリクエスト列 τ に対して $ALG(\tau) \leq \alpha \cdot OPT(\tau)$ となる ALG が存在するならば $(2\alpha + 1)$ -競合的なアルゴリズムが存在する

証明 リクエスト r_i が到着したら, 最も近いサーバ (利用可能とは限らない) s_i に移動し, あとは s_i に生じた際の ALG の割り当てに従って割り当てるアルゴリズムを考える.

$\sigma = r_1 \cdots r_n$ に対して r_i の最適な割り当て先を s_i^* とすると以下のように示せる:

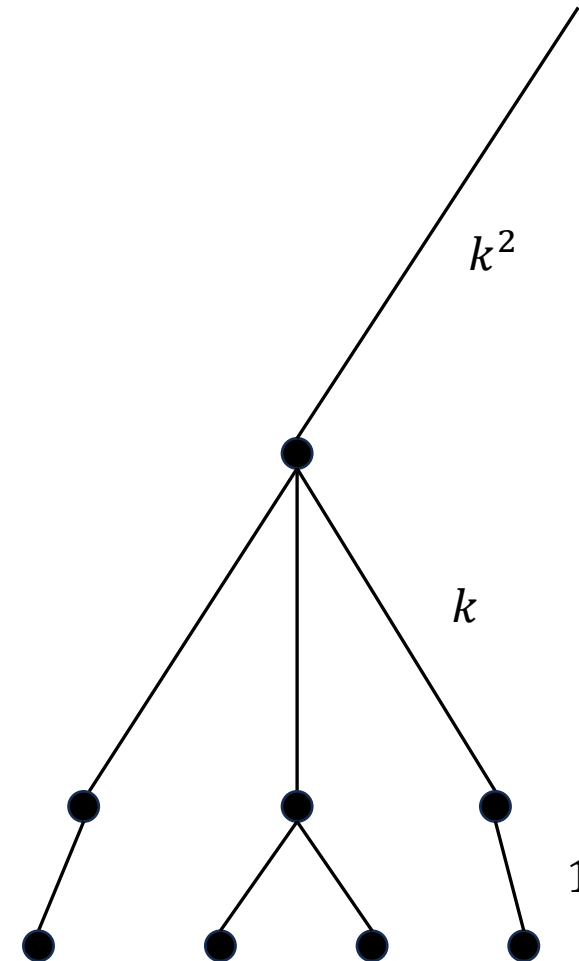
$$OPT(\tau) \leq \sum_i d(s_i, s_i^*) \leq \sum_i d(r_i, s_i^*) + \sum_i d(r_i, s_i) \leq 2 \cdot OPT(\sigma)$$

$$\therefore ALG(\sigma) \leq \sum_i d(r_i, s_i) + ALG(\tau) \leq OPT(\sigma) + \alpha \cdot OPT(\tau) \leq (2\alpha + 1)OPT(\sigma).$$

k -HST (Hierarchically well-Separated Tree)

重み付きの根付き木で次を満たすもの

- すべての葉が同じ深さにある
- 任意の頂点 v に対して, v とその子を接続する辺の重みは等しい
- 任意の頂点 v に対して親と接続する辺の重みが子と接続する辺の重みの k 倍



距離空間のHSTへの埋め込み

定理 [Fakcharoenphol, Rao and Talwar, 2004]

任意の n 点距離空間 (X, d_X) に対して, 2-HST距離空間の族 $\{(T, d_T)\}_{T \in \mathcal{T}}$ への確率的な埋め込み f が存在して次を満たす:

- 任意の $T \in \mathcal{T}$ と $x \in X$ について, $f(x)$ は 2-HST T 上の葉である.
- 任意の $T \in \mathcal{T}$ と $x, y \in X$ について, $d_X(x, y) \leq d_T(f(x), f(y))$.
- 任意の $x, y \in X$ について, $\mathbb{E}_T[d_T(f(x), f(y))] \leq O(\log n) d_X(x, y)$.

任意の n 点距離空間は 2-HST の葉ノードからなる距離空間に (確率的に) 埋め込んでも距離が高々 $O(\log n)$ 倍にしかない

→2-HSTに対してだけ乱択アルゴリズムを設計するという方針が有効

※確率的な埋め込みなので, 決定的アルゴリズムには使えない

問題の単純化

結果 $OMM(n)$ に対する乱択アルゴリズムの設計は次のように単純化される

系

- 葉ノード上にサーバが設置され,
- 葉ノード上にしかリクエストが到着しない
- 2-HST 上の

$OMM(n)$ に対して α -競合的なアルゴリズムが存在するならば,
一般の $OMM(n)$ に対して $O(\alpha \log n)$ -競合的な乱択アルゴリズムが存在する

[Bansal et al., 2007] の競合比 $O(\log^2 n)$ は 2-HST に対する $O(\log n)$ 競合的なアルゴリズムを構築することで達成された

一方, 2-HST に対する競合比下界も $\Omega(\log n)$ のためここで行き詰まり
現在に至るまで他に有効なアプローチは登場していない

乱択アルゴリズムまとめ

- 距離空間を良く整った木構造を持つ HST に埋め込みをする手法が有用
- その手法を用いて $O(\log^2 n)$ -競合的な乱択アルゴリズムを得られる
- 決定的アルゴリズムの競合比 $2n - 1$ よりも格段に改善（モデルは緩和しているが）

課題

- 競合比の上界 $O(\log^2 n)$ と下界 $\Omega(\log n)$ が不一致（未解決）
- HST への埋め込み手法では $O(\log^2 n)$ より良い競合比は得られないので、新しい設計手法か下界の改善が求められる

Robust-Matching アルゴリズム (RM)

Permutation アルゴリズムの一般化 [Raghvendra, 2016]

ϕ の代わりにパラメータ $t \geq 1$ を用いて ϕ_t を最小化する増加道 P_i を用いる

$$\phi_t(P_i) := t \cdot \sum_{(r,s) \in P_i \setminus M_{i-1}^*} d(r,s) - \sum_{(r,s) \in P_i \cap M_{i-1}^*} d(r,s)$$

特徴

- $t = 1$ なら Permutation に一致 (競合比 $2n - 1$)
- (t を定数扱いするなら) 任意の t に対して競合比 $O(n)$
- $t \rightarrow \infty$ のときも最善の競合比 $2n - 1$ に漸近
- 他の様々な設定でも柔軟に最適性を達成 (一部は後述)

RM のパラメータの直感的意味

$$\phi_t(P_i) := t \cdot \sum_{(r,s) \in P_i \setminus M_{i-1}^*} d(r,s) - \sum_{(r,s) \in P_i \cap M_{i-1}^*} d(r,s)$$

仮想的割り当て M_i^* を求める際に以下をバランスする

1. 「付け替え」をどれくらい嫌うか

t が大きいほど辺の本数が少ない増加道を選びやすくなる

2. そのトレードオフとして最適性をどのくらい犠牲にするか

M_i^* は Permutation の場合は $r_1, \dots, r_i$ の最適割り当てだったが, RMでは最適割り当てコストの t 倍以下という保証になる

Robust-Matching 再訪 1

[Nayyar and Raghvendra, 2017] は

- RM アルゴリズムの競合比が $O(\mu_{\mathcal{M}}(S)\log^2 n)$ であること
- 任意の決定的アルゴリズムの競合比が $\Omega(\mu_{\mathcal{M}}(S))$ であることを示した.

$$\mu_{\mathcal{M}}(S) := \frac{\text{TSP}(S)}{\text{diam}(S)}$$

$\text{TSP}(S)$: すべてのサーバを訪れて元の地点に戻る最短経路長

$\text{diam}(S)$: 異なるサーバ間の距離の最大値

→ サーバ数 n よりも $\mu_{\mathcal{M}}(S) \leq n$ の方が本質的なパラメータなのでは

→ $\mu_{\mathcal{M}}(S)$ が $\log n$ より十分大きいオーダーなら RM は最善に近い

Robust-Matching 再訪 2

$\mu_{\mathcal{M}}(S)$ がどんな値になるか？

- $\mu_{\mathcal{M}}(S) \leq n$ は常に成立する
- $\mathcal{M}$ の doubling 次元が d なら $\mu_{\mathcal{M}}(S) = O(n^{1-\frac{1}{d}})$

Doubling 次元が d であるとは:
任意の r について
半径 r の円を半径 $r/2$ の円 2^d 個
で覆えること

距離空間	$\mu_{\mathcal{M}}(S)$	競合比のオーダー
直線	2	$\log^2 n$
$\mathbb{R}^d$	$O(n^{1-\Theta(\frac{1}{d})})$	$n^{1-\Theta(\frac{1}{d})} \log^2 n$
一様距離空間	n	$n \log^2 n$

オンライン輸送問題（その他の研究）

乱択アルゴリズム

- $O(\log^2 m)$ -競合的な乱択アルゴリズム [Kalyanasundaram et al., 2023]

➤ 乱択アルゴリズムの競合比の上下界は OMM 同様未解決

容量 c_i/k しか持たない場合の最適解を OPT とするモデル

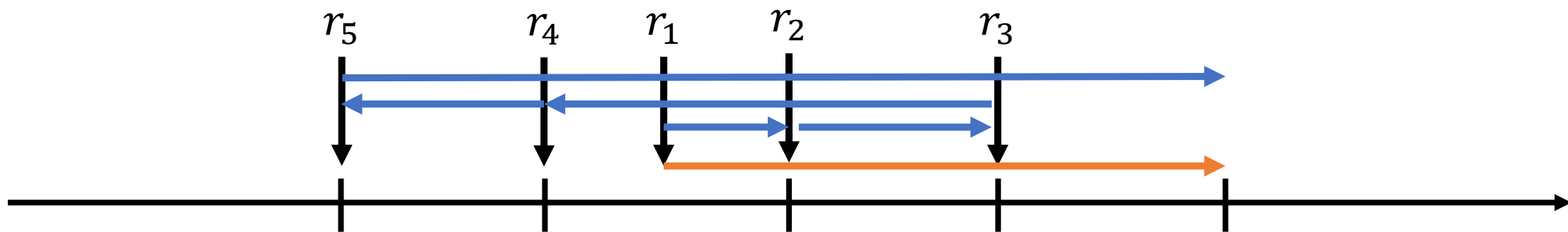
- $k = 2$ のとき $O(1)$ -競合的な決定的アルゴリズム [Kalyanasundaram and Pruhs, 2000]
- $k \geq 3$ のとき貪欲法が $\left(1 + \frac{2}{k-2}\right)$ -競合的 [Arndt et al., 2023]

一つ減らした容量 $c_i - 1$ を持つ場合の最適解を OPT とするモデル

- $O(\log m)$ -競合的な決定的アルゴリズム [Chung et al., 2008]

直線上のOMM 1

当初 9-競合的アルゴリズムの存在を予想されていた
[Kalyanasundaram and Pruhs, 1996]



「直前のリクエストで埋まったサーバのすぐ上に次のリクエストを置く」
これが最悪のシナリオであると考えられていた

このようなリクエスト列しか来ない場合（Cow-Path Problem と呼ばれる）は
9-競合的な決定的アルゴリズムが知られている

直線上のOMM 2

直線上のOMMに関する重要な研究

文献	競合比上界	競合比下界
[Fuchs et al., 2005]	-	9.001
[Antoniodis et al., 2014]	$o(n)$ (決定的)	-
[Nayyar and Raghvendra, 2017]	RMは $O(\log^2 n)$ -競合的	-
[Antoniodis et al., 2017]	-	自然なアルゴリズムは $\Omega(\log n)$
[Raghvendra, 2018]	RMは $O(\log n)$ -競合的	-
[Peserico and Scquizzato, 2021]	-	$\Omega(\sqrt{\log n})$

現状最善の競合比上界は $O(\log n)$ で下界は $\Omega(\sqrt{\log n})$
このギャップを埋めることは重要な未解決問題

$$B(x, r) = \{y : d(x, y) < r\}$$

Doubling 距離空間における OMM 1

[Gupta and Lewi, 2012] は Doubling 距離空間に対して $O(\log n)$ -競合的な乱択アルゴリズム (Random-Subtree) を提案した。

距離空間 $\mathcal{M} = (X, d)$ が doubling: 正の整数 λ が存在して次を満たすこと:
 任意の $x \in X$ と $r > 0$ に対して, ある $y_1, \dots, y_\lambda \in X$ が存在して

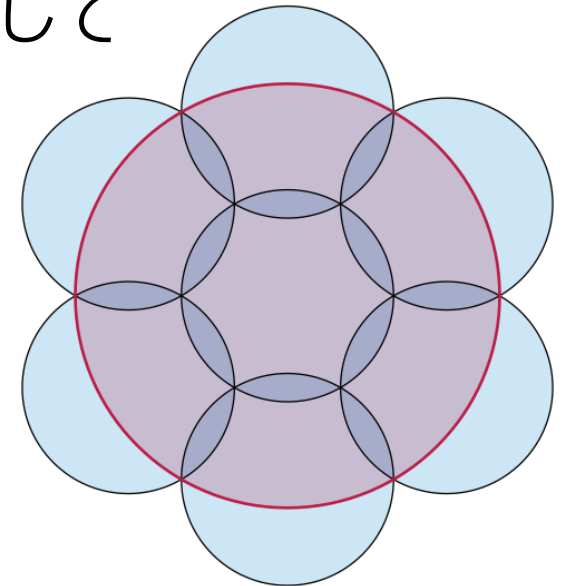
$$\bigcup_{i=1}^{\lambda} B\left(y_i, \frac{r}{2}\right) \subseteq B(x, r).$$

$\log_2 \lambda$ を $\mathcal{M}$ の doubling 次元という。

実際の競合比は $O(\text{poly}(\text{dim}_D) \log n)$

- 初めて競合比が $O(\log^2 n)$ を下回ったアルゴリズム

※ Doubling 距離空間での競合比下界は $\Omega(\log n)$ ではないのでタイトではない



https://en.wikipedia.org/wiki/Doubling_space

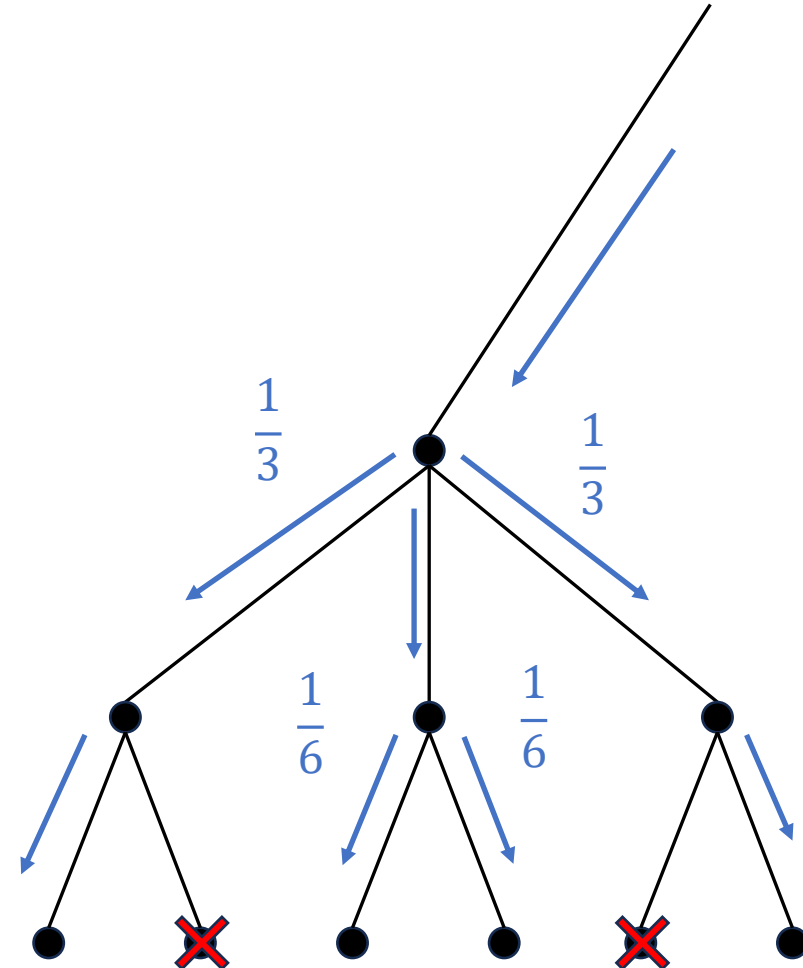
Doubling 距離空間における OMM 2

アルゴリズム (HST 上)

- 未使用サーバを子孫に含む最も低い祖先 a に移動する
- a の部分木の内, 未使用サーバを含むものを一様ランダムに選ぶ
- 選ばれた部分木内で同様に繰り返す

ポイント

- 単純な乱択貪欲法 (貪欲法でタイブレークを一様ランダムにする)
[Meyerson et al., 2006] ではない
- 空きの多い部分木に集まる確率が集中するのを防止



確率的 OMM (Stochastic OMM)

リクエストが**既知の確率分布**に従って i.i.d. で生成されるモデル
競合比はリクエストの分布とアルゴリズムの乱数に関する期待値で測る

[Gupta et al., 2019] は以下を満たすアルゴリズム (Fair-Bias) を提案：

- 一般の距離空間に対して $O((\log \log \log n)^2)$ -競合的
- 木に対して 9-競合的

定数競合比ができると予想されていて近年注目されている

- [Yang and Yu, 2025] は 3 次元以上のユークリッド空間で定数競合比を達成

ランダム到着モデル

リクエスト到着場所の集合はアルゴリズムを見て意地悪に決めることができるが、到着順は一様ランダムにしか出せない

動機：リクエストが未知の確率分布に従うモデルを扱いたいのが扱いにくい。それより強いが理論上扱いやすいモデルとして有名。

[Raghvendra, 2016] は以下を示し、決定的アルゴリズムの競合比を確定

- RM アルゴリズムが $(2H_n - 1 + o(1))$ -競合的であること
- 任意の決定的アルゴリズムの競合比が $2H_n - 1 - o(1)$ 以上であること

RM アルゴリズムは通常の設定 + ランダム到着モデルの双方で最適！

予測付きモデル

最適割り当て OPT に関する「予測」を途中で利用可能なモデル
一貫性と頑健性を両立する予測の活用方法を探す

一貫性 (consistency) 予測が完全に合っているなら定数競合比

頑健性 (robustness) 予測が的外れなときでも既存の競合比と同じオーダー

機械学習・AIの流行を背景として近年活発に研究されている
OMM へ適用する例も登場してきている

OMM への適用例と課題

η : 予測誤差 (0のときに完全に正しい予測)

[Antoniadis et al., 2023]

予測 : リクエスト r_i を受け取ったときに, OPT における $r_1, \dots, r_i$ の割当先の予測 P_i

競合比 : $O\left(\min\left\{\alpha, 1 + \frac{\eta}{OPT}\right\}\right)$

[Yang and Yu, 2025]

予測 : 全リクエストの到着場所の予測 R' を最初に得る

競合比 : $O\left(\min\left\{\alpha, 1 + \frac{\eta(\alpha+1)}{OPT}\right\}\right)$

α は既存の OMM に対するアルゴリズムが達成する競合比

今後の課題

- 更なる実用的な予測モデルとそれにふさわしい誤差 η の定義を与える
- $\frac{\eta}{OPT}$ への線形依存が改善できるか? (他の問題では改善される例がある)

まとめ

- OMM は患者の割り当てを抽象化した重要な問題
- OMM は未解決問題がたくさんある研究しがいのあるテーマ
- 近年は様々な新しいモデルが提案されている

重要な未解決問題

- 乱択アルゴリズムの競合比の上下界改善 $O(\log^2 n)$ vs $\Omega(\log n)$
- 直線上の OMM における競合比の上下界改善 $O(\log n)$ vs $\Omega(\sqrt{\log n})$
- 確率的 OMM における競合比の上下界改善 $O((\log \log \log n)^2)$ vs 定数

その他のテーマ

償還付きOMM (OMM with Recourse) [Gupta et al., 2019] [Megow and Nölke, 2025]

リクエストの付け替えを許し, 付け替え回数と競合比の関係を研究

- Permutation を修正すると, $O(\log n)$ 回の付け替えで $O(\log n)$ -競合的とできる
- 直線上の場合, $O(\log n)$ 回の付け替えで $O(1)$ -競合的とできる

オンラインボトルネックマッチング [Anthony and Chung, 2014]

リクエストとサーバ間の総和ではなく最大距離を最小化する

- 決定的アルゴリズムはやはり Permutation が (ほぼ) 最善で競合比 $O(n)$

少し関連度が低いがよく研究されているもの

Minimum-cost Perfect Matching with Delays: MPMD

遅延付きオンラインマッチング [Emek et al., 2016]

リクエスト同士のマッチングを求める. すぐにマッチ相手を決定する必要はないが, マッチ相手が決まるまでの時間がコストとして加算される.

文献リスト 1

- Antonios Antoniadis, Neal Barcelo, Michael Nugent, Kirk Pruhs, and Michele Scquizzato. A $o(n)$ -competitive deterministic algorithm for online matching on a line. WAOA 2014.
- Barbara M. Anthony and Christine Chung. Online bottleneck matching. *Journal of Combinatorial Optimization* 27.1 (2014): 100-114.
- Antonios Antoniadis, Christian Coester, Marek Eliáš, Adam Polak, and Bertrand Simon. Online metric algorithms with untrusted predictions. *ACM transactions on algorithms* 19.2 (2023): 1-34.
- Antonios Antoniadis, Carsten Fischer, and Andreas Tönnis. A collection of lower bounds for online matching on the line. LATIN 2018.
- Nikhil Bansal, Niv Buchbinder, Anupam Gupta, and Joseph Naor. An $O(\log^2 k)$ -competitive algorithm for metric bipartite matching. ESA 2007.
- Christine Chung, Kirk Pruhs, and Patchrawat Uthaisombut. The online transportation problem: On the exponential boost of one extra server. LATIN 2008.
- Yuval Emek, Shay Kutten, and Roger Wattenhofer. Online matching: haste makes waste! STOC 2016.
- Anupam Gupta, Guru Guruganesh, Binghui Peng, and David Wajc. Stochastic online metric matching. ICALP 2019.
- Varun Gupta, Ravishankar Krishnaswamy, Sai Sandeep. Permutation Strikes Back: The Power of Recourse in Online Metric Matching. APPROX/RANDOM 2020.

文献リスト 2

- Anupam Gupta and Kevin Lewi. The online metric matching problem for doubling metrics. ICALP 2012.
- Tsubasa Harada and Toshiya Itoh. A Nearly Optimal Deterministic Algorithm for Online Transportation Problem. ICALP 2025.
- Bala Kalyanasundaram and Kirk Pruhs. Online weighted matching. J. Algorithms, 14(3):478-488, 1993.
- Bala Kalyanasundaram and Kirk Pruhs. On-line network optimization problems. Developments from a June 1996 seminar on Online algorithms: the state of the art, pages 268–280, 1998.
- Bala Kalyanasundaram and Kirk Pruhs. On-line network optimization problems. Developments from a June 1996 seminar on Online algorithms: the state of the art, pages 268–280, 1998.
- Bala Kalyanasundaram, Kirk Pruhs, and Clifford Stein. A randomized algorithm for online metric b-matching. Oper. Res. Lett., 51(6):591–594, 2023.
- Samir Khuller, Stephen G. Mitchell, and Vijay V. Vazirani. On-line algorithms for weighted bipartite matching and stable marriages. Theor. Comput. Sci., 127(2):255–267, 1994.
- Nicole Megow and Lukas Nölke. Online Metric Matching on the Line with Recourse. *Algorithmica* 87, 813–841 (2025).

文献リスト 3

- Adam Meyerson, Akash Nanavati, and Laura J. Poplawski. Randomized online algorithms for minimum metric bipartite matching. SODA 2006.
- Krati Nayyar and Sharath Raghvendra. An input sensitive online algorithm for the metric bipartite matching problem. FOCS 2017.
- Enoch Peserico and Michele Scquizzato. Matching on the line admits no $o(\sqrt{\log n})$ -competitive algorithm. ACM Trans. Algorithms, 19(3):28:1–28:4, 2023.
- Sharath Raghvendra. A robust and optimal online algorithm for minimum metric bipartite matching. APPROX/RANDOM 2016.
- Sharath Raghvendra. Optimal analysis of an online algorithm for the bipartite matching problem on a line. SoCG 2018.
- Mingwei Yang and Sophie H. Yu. Online metric matching: Beyond the worst case. *Operations Research* 74.1 (2026): 130-140.